LAPORAN PROYEK UTS
PEMBELAJARAN MENDALAM



Judul : Klasifikasi Keberhasilan Pemasaran Bank Menggunakan Multi-Layer Perceptron (MLP)

Mata Kuliah : Pembelajaran Mendalam

Semester : Ganjil

TA. : 2026-2027

Nama Kelompok : Capcay

NIM & Nama Anggota Kelompok:

No.	NIM	Nama
1	231110273	Febri Sujaya
2	231110613	Andre Derizhy
3	231111378	Fandy Sanusi
4	231110663	Vincent Arfan
5	231112986	Bryan Lyanto

FAKULTAS INFORMATIKA

PROGRAM STUDI TEKNIK INFORMATIKA

UNIVERSITAS MIKROSKIL

2026

DAFTAR ISI

DAFTAR ISI ii
DAFTAR GAMBAR iv
DAFTAR TABEL vi
PEMBAGIAN TUGAS vii
A. Laporan Teknis Proyek 1
1. Kompleksitas Masalah 1
a. Mengapa masalah ini dianggap kompleks? 1
b. Apa tantangan utama dalam menyelesaikan masalah ini? 3
c. Apa saja pengembangan dan penyempurnaan dari Tugas Proyek 1? 4
2. Dataset yang Digunakan 6
a. Sumber dataset 6
b. Jumlah data, kelas/label, format, dan ukuran data 6
c. Apakah dataset bersifat real-world atau sintetis? 7
d. Seberapa kompleks dataset tersebut? 7
3. Exploratory Data Analysis (EDA) 8
a. Distribusi label/kelas 8
b. Histogram fitur numerik 9
c. Korelasi antarfitur 11
d. Contoh sampel data per kelas 12
e. Temuan penting EDA yang memengaruhi preprocessing 13
4. Preprocessing Data 13
a. Pembersihan data 13
b. Normalisasi/standardisasi dan encoding 16
c. Augmentasi data 18
d. Missing value, outlier, dan ketidakseimbangan kelas 18
e. Pembagian data dan pencegahan kebocoran 20

5. Model yang Digunakan 21
a. Jenis model 21
b. Arsitektur model 21
c. Konfigurasi pelatihan 24
d. Alasan pemilihan model dan arsitektur 26
e. Dua fitur unik 27
6. Hasil Evaluasi 31
a. Metrik evaluasi dan alasannya 31
b. Nilai metrik pada data testing 33
c. Grafik loss dan akurasi training vs validation 35
d. Perbandingan antarskenario 37
e. Analisis hasil 41
7. Teknologi yang Digunakan 43
8. Deployment dan Demo 44
a. Link deployment 44
b. Tahapan deployment 45
c. Penjelasan demo dan deployment 46
9. Lampiran 52
a. Code 52
b. Grafik Evaluasi 72
10. Daftar Pustaka 75
B. Tautan 77

DAFTAR GAMBAR

Gambar 1 Daftar Dataset 2

Gambar 2 Tipe Data pada Dataset 3

Gambar 3 Missing Value dan Target pada Dataset 3

Gambar 4 Grafik Distribusi Kelas Target 9

Gambar 5 Histogram Fitur Numerik 9

Gambar 6 Korelasi Antar Fitur 12

Gambar 7 Contoh Sampel Data: Proporsi Target Yes 12

Gambar 8 Code Pembersihan Data 15

Gambar 9 Output Code Pembersihan Data 16

Gambar 10 Preprocessing Data 17

Gambar 11 Perhitungan Class Weight 19

Gambar 12 Output Code Pembagian Data 20

Gambar 13 Arsitektur Model Baseline 22

Gambar 14 Arsitektur Model Tuned 23

Gambar 15 Permutation Importance 30

Gambar 16 Ablation Study 30

Gambar 17 Confusion Matrix Model MLP Tuned pada Data Testing 34

Gambar 18 Grafik Loss dan Accuracy Curve pada Model Baseline dan Tuned 36

Gambar 19 Grafik Permutation Importance pada Model Tuned 42

Gambar 20 Struktur dan Isi Folder Artifacts 45

Gambar 21 Tampilan Antarmuka Aplikasi Bank Deposit Predictor 46

Gambar 22 Contoh Input Aplikasi Bank Deposit Predictor 48

Gambar 23 Output Hasil Prediksi Aplikasi Bank Deposit Predictor 49

DAFTAR TABEL

Tabel 1 Pembagian Tugas Anggota Kelompok x

Tabel 2 Ringkasan Dataset 7

Tabel 3 Tabel Pembagian Data 20

Tabel 4 Tabel Arsitektur Model Baseline 23

Tabel 5 Tabel Arsitektur Model Tuned 24

Tabel 6 Konfigurasi Pelatihan 25

Tabel 7 Fungsi dan Alasan Penggunaan Metrik 32

Tabel 8 Hasil Data Testing 33

Tabel 9 Perbandingan Metrik Class Weighting dan Threshold Tuning antar Skenario 38

Tabel 10 Perbandingan Metrik Threshold Tuning 39

Tabel 11 Hasil Akhir Data Testing 39

Tabel 12 Teknologi dan Versi yang digunakan 44

Tabel 13 Daftar Tautan 77

PEMBAGIAN TUGAS

No.	NIM	Nama	Peran	Rincian Pekerjaan
1	231110273	Febri Sujaya	Pengolahan Data dan Penyusunan Laporan	1. Membantu mengembangkan kode program awal untuk proses preprocessing dan pengolahan dataset sebelum digunakan dalam pelatihan model. 2. Membantu melakukan pemeriksaan dan persiapan data agar dapat digunakan pada tahap pengembangan model Deep Learning. 3. Membantu menyusun, melengkapi, dan merapikan bagian laporan, khususnya yang berkaitan dengan preprocessing dan tahapan pengolahan data. 4. Membantu melakukan pemeriksaan kesesuaian penjelasan laporan dengan tahapan implementasi program. 5. Mencarikan beberapa referensi untuk daftar pustaka dan peletakan sitasi.
2	231110613	Andre Derizhy	Penyusunan dan Review Laporan	1. Membantu menyusun, melengkapi, dan merapikan isi laporan

				tugas. 2. Melakukan review laporan dengan memperbaiki kesalahan kata, typo, dan kesalahan penginputan angka data. 3. Memeriksa konsistensi penulisan, format tabel, serta kesesuaian antarbagian laporan. 4. Membantu memastikan laporan tersusun secara sistematis dan mudah dipahami. 5. Mencarikan beberapa referensi untuk daftar pustaka dan peletakan sitasi.
3	231111378	Fandy Sanusi	Pengembangan Model dan Penyusunan Laporan	1. Mengembangkan dan mengimplementasikan kode program untuk proses pelatihan model MLP, mulai dari model baseline hingga model tuned. 2. Melakukan pengaturan dan eksperimen terhadap konfigurasi model serta teknik optimasi untuk meningkatkan performa model. 3. Melakukan evaluasi dan analisis hasil pelatihan model berdasarkan metrik

				evaluasi yang digunakan. 4. Membantu menyusun, melengkapi, dan merapikan isi laporan tugas. 5. Mengelola referensi daftar pustaka menggunakan Mendeley.
4	231110663	Vincent Arfan	Pengembangan Ide Proyek, Pengelolaan Repository, dan Deployment Aplikasi	1. Mencari topik yang akan dilakukan untuk pengembangan dan pengimplementasikan Model Deep Learning. Dan kemudian disetujui oleh teman teman sekelompok. 2. Mencari Dataset yang akan digunakan untuk pengembangan model deep learning. Dan kemudian disetujui oleh teman teman sekelompok. 3. Membantu penyusunan laporan dalam tahap evaluasi dari hasil pengembangan model, library yang digunakan. 4. Melakukan capture hasil code pada lampiran laporan, dan menyesuaikan isi laporan dengan code. 5. Membuat dan mengelola

				repository GitHub dan melakukan deploy aplikasi di Streamlit
5	231112986	Bryan Lyanto	Dokumentasi Proyek dan Repository	1. Membantu merapikan struktur folder dan file proyek agar tersusun secara sistematis di dalam repository GitHub. 2. Menyusun dan melengkapi file README.md yang berisi deskripsi proyek, tujuan pengembangan, serta informasi mengenai dataset dan model yang digunakan.

Tabel 1 Pembagian Tugas Anggota Kelompok

A. Laporan Teknis Proyek

1. Kompleksitas Masalah

Kegiatan telemarketing merupakan salah satu strategi pemasaran yang digunakan oleh bank untuk menawarkan produk deposito berjangka kepada calon nasabah. Namun, keberhasilan kampanye pemasaran tidak selalu dapat dipastikan karena keputusan nasabah dipengaruhi oleh berbagai faktor, sehingga diperlukan pendekatan berbasis data untuk memprediksi kemungkinan keberhasilan kampanye tersebut [1]. Proyek ini berfokus pada prediksi keberhasilan kampanye pemasaran langsung (direct marketing campaigns) yang dilakukan oleh lembaga perbankan melalui media komunikasi telepon (phone calls). Hal utama yang diprediksi adalah peluang atau keputusan seorang calon nasabah untuk menerima dan berlangganan produk deposito berjangka (term deposit), yang diekstrak dari data demografi, riwayat kampanye, serta kondisi ekonomi nasabah. Sistem prediktif ini ditujukan secara langsung bagi tim manajemen dan divisi pemasaran (marketing team) bank guna membantu mereka mengenali kelompok nasabah yang memiliki probabilitas konversi tertinggi.

Dari sudut pandang bisnis, penerapan model berbasis Multi-Layer Perceptron (MLP) ini memberikan nilai strategis yang signifikan untuk efisiensi operasional perbankan. Dengan kemampuan menyaring dan menargetkan calon nasabah secara lebih presisi, institusi keuangan dapat menghindari pemborosan biaya dan waktu akibat menghubungi nasabah yang tidak potensial, sekaligus meningkatkan tingkat konversi kampanye (conversion rate) dan memaksimalkan perolehan dana pihak ketiga secara efektif.

a. Mengapa masalah ini dianggap kompleks?

Permasalahan prediksi pemasaran ini dikategorikan kompleks karena melibatkan tantangan struktural dan karakteristik data dunia nyata (real-world tabular data), yang dapat diuraikan melalui beberapa aspek berikut:

1. Ketimpangan Kelas yang Ekstrem (Class Imbalance)
   Jumlah nasabah yang menolak penawaran jauh lebih mendominasi dibandingkan nasabah yang benar-benar mau berlangganan deposito. Berdasarkan eksplorasi data, kelas negatif (tidak berlangganan) mencapai 88,30%, sementara kelas positif (berlangganan) hanya 11,69%. Model berisiko tinggi mengalami bias, yakni

cenderung menebak kelas mayoritas demi mengejar tingkat akurasi semu, namun gagal mendeteksi kelas minoritas yang menjadi target utama bisnis.

2. Kombinasi Fitur yang Beragam (Numerik dan Kategorikal)

   Dataset tidak hanya memuat variabel angka, tetapi juga atribut teks yang merepresentasikan latar belakang demografi dan profil nasabah. Terdapat 9 kolom kategorikal (seperti jenis pekerjaan, status pernikahan, dan tingkat pendidikan) yang harus diproses bersamaan dengan data numerik. Diperlukan tahapan prapemrosesan lanjutan (seperti One-Hot Encoding) agar format teks tersebut dapat diterjemahkan ke dalam representasi numerik yang kompatibel dengan arsitektur jaringan saraf tiruan.

3. Adanya Nilai Khusus yang Mewakili Kondisi Tertentu (Implicit Missing Value)

   Sejumlah kolom menyimpan nilai penanda khusus untuk merekam kondisi di mana suatu kejadian belum pernah terjadi sebelumnya pada riwayat nasabah. Pada kolom riwayat hari kontak terakhir (pdays), tercatat nilai minimum sebesar -1, yang mengindikasikan bahwa sebagian besar nasabah belum pernah dihubungi sama sekali pada kampanye sebelumnya. Nilai khusus ini memerlukan penanganan khusus agar tidak diinterpretasikan secara keliru sebagai besaran matematis murni yang dapat merusak proses pembelajaran model.

4. Sinyal Prediktif yang Lemah pada Sebagian Besar Fitur

   Secara terpisah, sebagian besar variabel awal memiliki hubungan linier yang sangat lemah terhadap keputusan akhir nasabah. Koefisien korelasi fitur numerik terhadap target tercatat sangat rendah, misalnya usia (age) sebesar 0,02, saldo (balance) sebesar 0,05, dan jumlah kontak (campaign) sebesar -0,07. Model tidak dapat mengandalkan variabel tunggal untuk mengambil keputusan, melainkan menuntut penggunaan arsitektur berlapis (Multi-Layer Perceptron) yang mampu merangkai pola non-linear dari berbagai sinyal lemah yang tersedia.

```
Ukuran dataset: (45211, 17)
Jumlah fitur asli: 16
Jumlah duplikat: 0
```

	age	job	marital	education	default	balance	housing	loan	contact	day_of_week	month	duration	campaign	pdays	previous	poutcome	target
0	58	management	married	tertiary	no	2143	yes	no	NaN	5	may	261	1	-1	0	NaN	0
1	44	technician	single	secondary	no	29	yes	no	NaN	5	may	151	1	-1	0	NaN	0
2	33	entrepreneur	married	secondary	no	2	yes	yes	NaN	5	may	76	1	-1	0	NaN	0
3	47	blue-collar	married	NaN	no	1506	yes	no	NaN	5	may	92	1	-1	0	NaN	0
4	33	NaN	single	NaN	no	1	no	no	NaN	5	may	198	1	-1	0	NaN	0

Gambar 1 Daftar Dataset

	dtype
age	int64
job	object
marital	object
education	object
default	object
balance	int64
housing	object
loan	object
contact	object
day_of_week	int64
month	object
duration	int64
campaign	int64
pdays	int64
previous	int64
poutcome	object
target	int32

Gambar 2 Tipe Data pada Dataset

	missing
age	0
job	288
marital	0
education	1857
default	0
balance	0
housing	0
loan	0
contact	13020
day_of_week	0
month	0
duration	0
campaign	0
pdays	0
previous	0
poutcome	36959
target	0

target	jumlah
No	39922
Yes	5289

Gambar 3 Missing Value dan Target pada Dataset

b. Apa tantangan utama dalam menyelesaikan masalah ini?

Penyusunan model prediktif pada dataset pemasaran perbankan ini dihadapkan pada beberapa tantangan teknis yang krusial, antara lain:

1. Risiko Kebocoran Data (Data Leakage) pada Fitur Durasi
   Tantangan terbesar dalam pemodelan ini adalah mengidentifikasi dan mengamankan model dari atribut yang menyimpan informasi masa depan (*future leakage*). Berdasarkan analisis eksplorasi data, kolom duration (durasi panggilan) memiliki korelasi artifisial yang sangat tinggi terhadap target. Padahal, durasi percakapan baru dapat diketahui *setelah* interaksi pemasaran selesai dilakukan. Jika fitur ini diikutsertakan, model akan terlihat sangat akurat saat pelatihan, namun gagal total saat diterapkan secara nyata di lapangan. Melakukan pembersihan data dengan mengecualikan atau menghapus kolom duration sebelum tahap pelatihan model.
2. *Trade-off* antara *Precision* dan *Recall* Akibat Ketimpangan Kelas
   Akibat ketimpangan data yang ekstrem (hanya 11,69% kelas positif), model rentan mengalami bias yang memicu dilema antara ketepatan tebakan (*precision*) dan kelengkapan tangkapan nasabah potensial (*recall*). Jika evaluasi hanya mengandalkan akurasi global, model dapat menebak seluruh kelas sebagai "tidak berlangganan" dan tetap menghasilkan akurasi tinggi (~88%), namun nilai *recall* untuk kelas minoritas akan menjadi nol (gagal mendeteksi target bisnis). Penerapan teknik penyesuaian bobot kelas (*Class Weights*) agar model memberikan penalti yang lebih besar pada kesalahan prediksi di kelas minoritas.
3. Risiko *Overfitting* pada Pelatihan Jaringan Saraf Tiruan (*Multi-Layer Perceptron*)
   Arsitektur *Multi-Layer Perceptron* (MLP) yang fleksibel memiliki kapasitas tinggi untuk menghafal pola data latih (*overfitting*) alih-alih mempelajari pola generalisasi yang baik untuk data baru. Penerapan mekanisme regularisasi seperti *Dropout*, *Batch Normalization*, serta pemantauan proses pelatihan menggunakan *Early Stopping* agar model berhenti di titik optimal sebelum mengalami *overfitting*.

c. Apa saja pengembangan dan penyempurnaan dari Tugas Proyek 1?

Pada Tugas Proyek 1, pengembangan proyek telah mencakup proses eksplorasi dataset, preprocessing data, pembangunan model Multi-Layer Perceptron (MLP), serta evaluasi awal untuk mengklasifikasikan keberhasilan pemasaran bank. Pada

pengembangan proyek UTS, dilakukan sejumlah penambahan dan penyempurnaan untuk meningkatkan kelengkapan eksperimen, memperdalam evaluasi model, serta mempersiapkan model agar dapat digunakan melalui aplikasi.

Beberapa pengembangan dan penyempurnaan yang dilakukan adalah sebagai berikut:

1. Pengembangan Feature Engineering

   Pada proyek sebelumnya, dilakukan penambahan dua fitur turunan, yaitu pdays_contacted dan balance_per_age. Pada pengembangan UTS, ditambahkan dua fitur baru, yaitu pdays_recency_group dan campaign_intensity, sehingga jumlah fitur hasil rekayasa menjadi empat fitur.

2. Penambahan Eksperimen dan Perbandingan Model

   Eksperimen dikembangkan dengan membandingkan model MLP Baseline dan MLP Tuned, serta menambahkan perbandingan model Tuned dengan dan tanpa penerapan Class Weighting. Perbandingan dilakukan menggunakan metrik evaluasi pada validation set untuk menentukan konfigurasi model yang paling sesuai.

3. Penyempurnaan Pemilihan Threshold

   Pada proyek sebelumnya, pemilihan threshold difokuskan pada konfigurasi model tertentu. Pada UTS, proses threshold tuning dikembangkan untuk membandingkan beberapa kandidat model dan menentukan threshold berdasarkan nilai F1-score kelas Yes pada validation set.

4. Penambahan Analisis Interpretabilitas Model

   Pada UTS, ditambahkan analisis Permutation Importance untuk mengetahui kontribusi relatif setiap fitur terhadap performa prediksi model. Analisis ini membantu mengidentifikasi fitur-fitur yang paling berpengaruh terhadap hasil klasifikasi.

5. Penambahan Ablation Study

   Dilakukan eksperimen tambahan dengan membandingkan performa model ketika empat fitur hasil feature engineering digunakan dan ketika seluruh fitur tersebut dihilangkan. Pengujian ini bertujuan untuk mengetahui pengaruh gabungan fitur tambahan terhadap performa model.

6. Penyempurnaan Evaluasi dan Pemilihan Model Final

Evaluasi dikembangkan dengan menggunakan beberapa metrik, yaitu Accuracy, Precision, Recall, F1-score, ROC-AUC, dan PR-AUC. Pemilihan model final dilakukan berdasarkan hasil validation set, sedangkan test set digunakan untuk mengevaluasi performa akhir model terpilih.

7. Penyimpanan Model dan Artefak untuk Deployment
   Pada pengembangan UTS, ditambahkan proses penyimpanan model terpilih beserta artefak preprocessing, seperti scaler, daftar fitur, dan metadata preprocessing. Artefak tersebut digunakan untuk mendukung proses inferensi pada aplikasi Streamlit.

Secara keseluruhan, pengembangan UTS tidak hanya berfokus pada penambahan fitur, tetapi juga memperluas eksperimen, meningkatkan analisis performa model, serta mempersiapkan implementasi model agar dapat digunakan melalui aplikasi.

2. Dataset yang Digunakan

a. Sumber dataset

Dataset yang digunakan dalam penelitian/proyek ini bersumber dari UCI Machine Learning Repository dengan identifikasi dataset Bank Marketing (UCI ID: 222) [2]. Dataset ini dapat diakses secara publik melalui repositori resmi UCI Machine Learning Repository atau dimuat secara terprogram menggunakan pustaka Python ucimlrepo melalui fungsi fetch_ucirepo(id=222).

b. Jumlah data, kelas/label, format, dan ukuran data

Atribut	Keterangan
Versi dataset	Bank Marketing (UCI Machine Learning Repository, ID: 222)
Jumlah data	45.211 baris
Jumlah fitur awal	16 fitur input
Fitur yang dihapus	1 fitur, yaitu duration
Fitur tambahan	4 fitur feature engineering, yaitu pdays_contacted, balance_per_age, pdays_recency_group, dan campaign_intensity

Jumlah fitur sebelum encoding	19 fitur (15 fitur asli yang dipertahankan dan 4 fitur tambahan)
Jumlah fitur setelah encoding	44 fitur
Kelas / label	Biner: 0 = Tidak Berlangganan (No), 1 = Berlangganan Deposito (Yes)
Jumlah per kelas	No: 39.922 (88,30%); Yes: 5.289 (11,70%)
Format	Data Tabular
Tipe fitur pada fitur awal	7 Numerik (Integer/Float) & 9 Kategorikal (String/Object)

Tabel 2 Ringkasan Dataset

Dataset Bank Marketing memiliki 45.211 baris data dengan 16 fitur asli dan satu kolom target, yaitu y. Pada tahap preprocessing, fitur duration dihapus karena informasi durasi panggilan baru tersedia setelah komunikasi selesai, sehingga tidak sesuai untuk skenario prediksi yang dilakukan sebelum panggilan berlangsung.

Selanjutnya, dilakukan feature engineering dengan menambahkan empat fitur baru, yaitu pdays_contacted, balance_per_age, pdays_recency_group, dan campaign_intensity. Dengan demikian, jumlah fitur yang digunakan sebelum proses encoding menjadi 19 fitur.

Setelah dilakukan imputasi dan One-Hot Encoding terhadap fitur kategorikal, jumlah fitur meningkat menjadi 44 kolom. Seluruh fitur tersebut kemudian digunakan sebagai input dalam proses pelatihan model MLP.

c. Apakah dataset bersifat real-world atau sintetis?

Dataset ini murni bersifat data dunia nyata (*real-world data*). Data tersebut merekam jejak operasional asli dari serangkaian kampanye pemasaran langsung (*direct marketing campaigns*) berbasis komunikasi telepon yang dilakukan oleh lembaga perbankan komersial di Portugal [1]. Dari sudut pandang metodologi pengumpulan data, keaslian (*authenticity*) dataset ini dicirikan oleh adanya faktor manusia (*human behavior*) dan dinamika ekonomi eksternal yang melekat di dalamnya.

d. Seberapa kompleks dataset tersebut?

Dataset *Bank Marketing* memiliki kompleksitas pada atribut dan sebaran statistik, yang dicirikan oleh beberapa aspek intrinsik berikut:

1. Struktur Fitur yang Heterogen dan Berdimensi Luas
   Dataset ini memuat 16 kolom yang terpecah antara variabel numerik kontinu dan 9 atribut kategorikal dengan variasi kategori yang beragam. Kompleksitas struktural ini terlihat ketika atribut teks harus ditransformasikan, di mana penambahan variabel kategori akan memperluas dimensi ruang fitur secara signifikan setelah proses One-Hot Encoding.

2. Distribusi Nilai yang Sangat Skewed dan Rentang Ekstrem
   Secara statistik, variabel numerik dalam dataset ini memiliki tingkat kemiringan (skewness) yang tinggi dan tidak berdistribusi normal. Terdapat disparitas rentang nilai yang ekstrem antar-kolom, seperti fitur balance yang memiliki nilai negatif hingga ratusan ribu, serta atribut seperti campaign dan previous yang terkonsentrasi pada nilai kecil dengan sebaran ekor yang panjang.

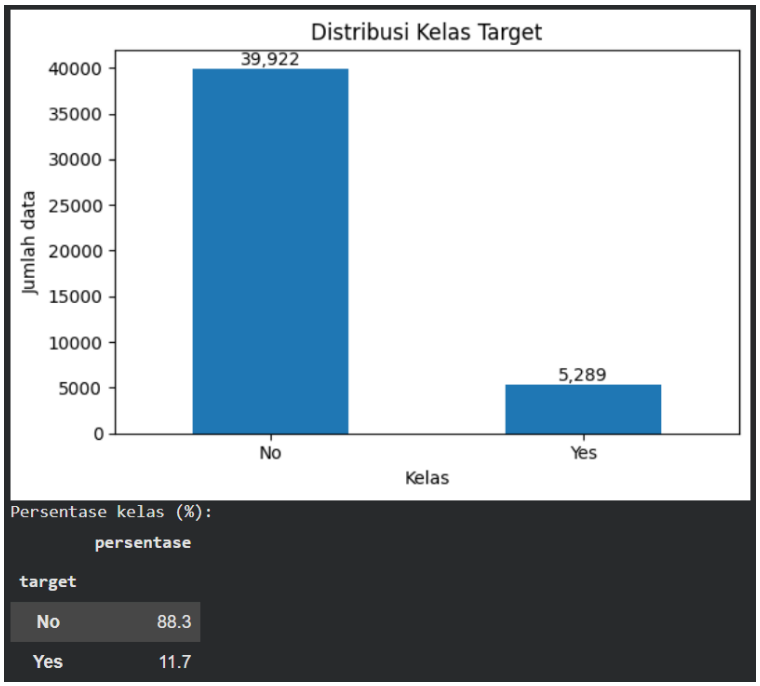
3. Artefak Struktural dan Nilai Penanda Khusus
   Format penyimpanan nilai di dalam dataset mencerminkan skema basis data operasional asli. Hal ini dibuktikan dengan adanya nilai penanda khusus—seperti penggunaan angka -1 pada kolom pdays—yang secara struktural merepresentasikan kondisi "belum pernah dikontak", sehingga memerlukan pemahaman skema data yang cermat agar tidak dibaca sebagai anomali matematis murni.

4. Distribusi Sampel yang Timpang pada Level Baris
   Pada tingkat komposisi data secara keseluruhan, dataset ini memiliki ketidakseimbangan proporsi baris yang melekat secara alami dari hasil rekaman kampanye industri, di mana mayoritas besar sampel terkonsentrasi secara masif pada kelas non-aktif (negatif) dibandingkan kelas target utama.

3. Exploratory Data Analysis (EDA)

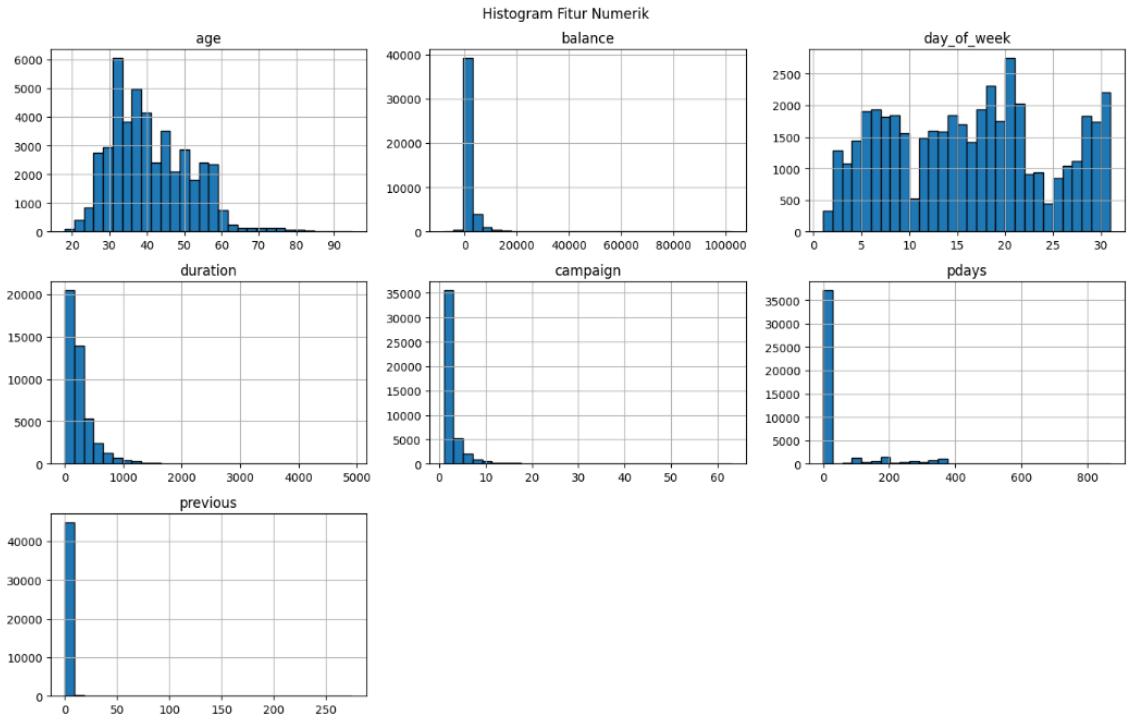
a. Distribusi label/kelas

Eksplorasi menunjukkan adanya ketidakseimbangan kelas yang signifikan. Mayoritas sampel terkonsentrasi pada kelas 0 (no) sebesar 88,30% (39.922 sampel), sedangkan kelas target 1 (yes) hanya mencakup 11,70% (5.289 sampel).



Gambar 4 Grafik Distribusi Kelas Target

b. Histogram fitur numerik



Gambar 5 Histogram Fitur Numerik

Gambar ini menunjukkan visualisasi *Exploratory Data Analysis* (EDA) berupa histogram untuk fitur-fitur numerik pada dataset Bank Marketing. Visualisasi ini memperlihatkan distribusi frekuensi (jumlah baris data) pada sumbu-y untuk setiap rentang nilai variabel pada sumbu-x.

Berikut adalah penjelasan pola distribusi untuk masing-masing fitur:

- age (Usia): Distribusi usia cenderung right-skewed (condong ke kanan). Mayoritas target kampanye nasabah berada pada kelompok usia produktif, yaitu terpusat antara 30 hingga 40 tahun. Frekuensinya semakin menurun secara bertahap pada rentang usia lanjut.
- balance (Saldo Rekening): Distribusi sangat right-skewed dengan ekor yang sangat panjang ke kanan. Sebagian besar nasabah memiliki saldo rekening di sekitar angka nol atau relatif rendah, namun terdapat outlier ekstrem dengan saldo sangat tinggi (mencapai nyaris 100.000).
- day_of_week (Tanggal Kontak Terakhir): Meskipun nama kolom pada dataset adalah day_of_week, nilai yang terdapat di dalamnya berada pada rentang 1 hingga 31. Oleh karena itu, fitur ini merepresentasikan tanggal dalam satu bulan ketika kontak terakhir dilakukan, bukan nama hari dalam seminggu. Histogram menunjukkan bahwa frekuensi kontak tersebar pada berbagai tanggal, dengan beberapa tanggal memiliki jumlah aktivitas kontak yang lebih tinggi dibandingkan tanggal lainnya.
- duration (Durasi Panggilan): Distribusinya condong ke kanan secara tajam. Mayoritas panggilan telepon berlangsung singkat (sebagian besar berada di bawah 500 detik atau sekitar 8 menit). Namun, terdapat outlier durasi panggilan yang sangat panjang, hingga melebihi 4000 detik.

  Kolom duration tetap disertakan pada tahap EDA untuk memahami karakteristik dan hubungannya dengan target. Namun, kolom ini tidak digunakan sebagai input model pada tahap pelatihan karena durasi panggilan hanya dapat diketahui setelah proses komunikasi selesai dilakukan. Hal ini dilakukan untuk menghindari data leakage dalam skenario prediksi keberhasilan pemasaran sebelum panggilan berlangsung.
- campaign (Jumlah Kontak selama Kampanye Ini): Sebagian besar nasabah hanya dihubungi sebanyak 1 hingga 3 kali. Ekor histogram memanjang jauh ke kanan,

menandakan adanya anomali di mana beberapa nasabah terus-menerus dihubungi hingga lebih dari 50 kali.

Selain digunakan sebagai fitur numerik asli, informasi pada kolom `campaign` juga dimanfaatkan dalam proses feature engineering untuk membentuk fitur campaign_intensity. Fitur tersebut mengelompokkan jumlah kontak selama kampanye berjalan menjadi tiga kategori, yaitu Low_1_contact, Medium_2_3_contacts, dan High_4plus_contacts. Pengelompokan ini bertujuan memberikan representasi kategorikal mengenai intensitas kontak yang diterima nasabah.

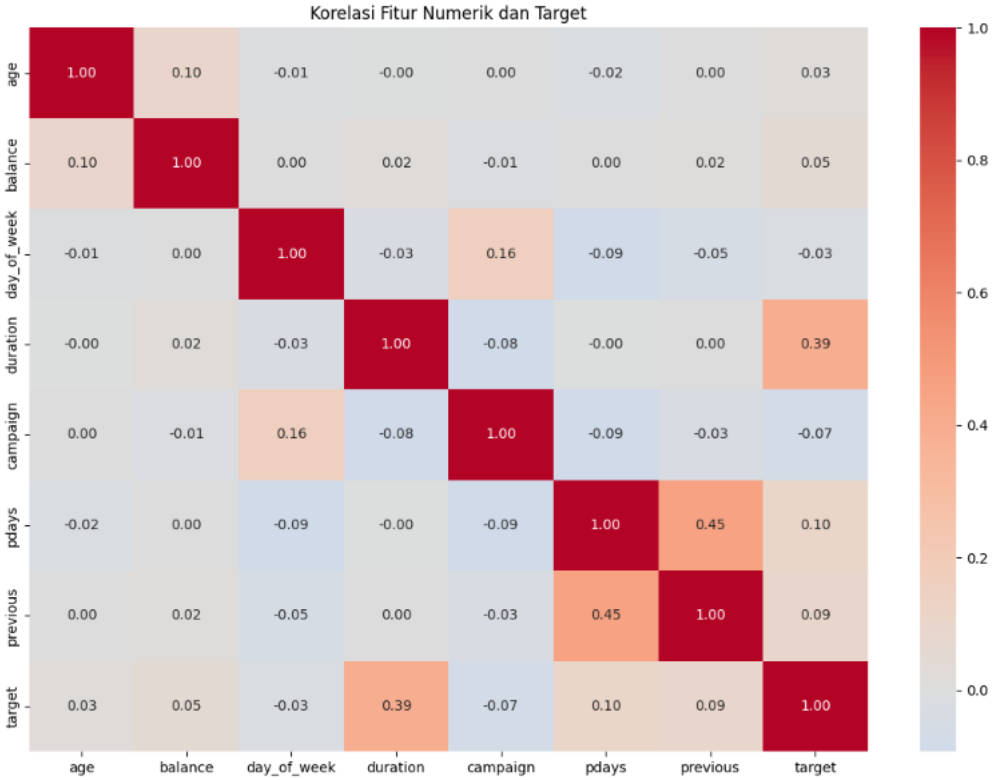
- pdays (Jarak Hari dari Kampanye Sebelumnya): Terdapat penumpukan nilai ekstrem pada batang paling kiri (nilai 0 atau -1). Hal ini menunjukkan bahwa mayoritas mutlak nasabah di dalam dataset ini merupakan prospek baru yang belum pernah dihubungi sama sekali pada kampanye-kampanye bank sebelumnya. Sisa data yang kecil tersebar hingga angka 800+ hari.
- previous (Jumlah Kontak Sebelum Kampanye Ini): Polanya berkorelasi kuat dengan pdays. Hampir seluruh baris data menumpuk di angka 0, yang kembali menegaskan bahwa mayoritas target adalah nasabah yang belum pernah memiliki riwayat kontak dari kampanye sebelumnya.

c. Korelasi antarfitur

Berikut adalah rincian informasi dari matriks tersebut:

- Korelasi dengan Target: Sesuai dengan narasi yang Anda susun sebelumnya, fitur duration menunjukkan korelasi positif tertinggi terhadap target dengan nilai 0.39. Fitur numerik lainnya memiliki korelasi yang sangat lemah terhadap target, secara berurutan: pdays (0.10), previous (0.09), balance (0.05), age (0.03), day_of_week (-0.03), dan campaign (-0.07).

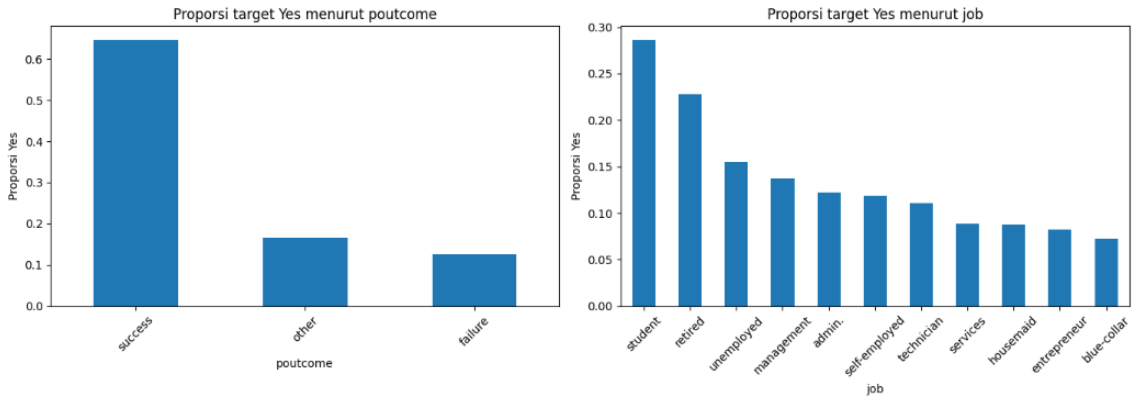
- Korelasi Antarfitur Independen: Hubungan terkuat antara sesama variabel prediktor terjadi antara fitur pdays dan previous dengan nilai korelasi 0.45. Matriks ini belum menunjukkan adanya gejala multikolinearitas yang ekstrem (korelasi di atas 0.8 atau 0.9) pada set data awal ini.



Gambar 6 Korelasi Antar Fitur

d. Contoh sampel data per kelas

Sampel kelas 1 (yes) cenderung memiliki riwayat keberhasilan kampanye sebelumnya (poutcome = success) serta saldo rata-rata yang lebih tinggi.



Gambar 7 Contoh Sampel Data: Proporsi Target Yes

Visualisasi ini memperlihatkan proporsi nasabah yang berlangganan deposito (target Yes) berdasarkan dua fitur kategorikal, yaitu poutcome dan job. Berdasarkan grafik, kategori success pada fitur poutcome menunjukkan proporsi keberhasilan berlangganan yang lebih tinggi dibandingkan kategori hasil kampanye sebelumnya lainnya. Hal ini

menunjukkan bahwa keberhasilan kampanye sebelumnya memiliki hubungan dengan peluang keberhasilan kampanye berikutnya.

Selain itu, proporsi target Yes juga berbeda pada setiap kategori pekerjaan. Perbedaan tersebut mengindikasikan bahwa karakteristik pekerjaan dapat memberikan informasi tambahan dalam mengenali kelompok nasabah yang memiliki potensi lebih tinggi untuk berlangganan deposito.

e. Temuan penting EDA yang memengaruhi preprocessing

- Dataset berisi 45.211 observasi dan 16 fitur asli.
- Target terdiri dari kelas No sebanyak 39.922 (sekitar 88,3%) dan kelas Yes sebanyak 5.289 (sekitar 11,7%).
- Distribusi target tidak seimbang. Karena itu, accuracy saja tidak cukup untuk menilai model; precision, recall, F1 kelas Yes, ROC-AUC, dan PR-AUC juga digunakan.
- Beberapa fitur kategorikal memiliki nilai kosong. Imputasi akan dipelajari dari training set untuk mencegah data leakage.

4. Preprocessing Data

a. Pembersihan data

- Penghapusan Fitur duration: Kode menyalin dataset mentah ke variabel X lalu menghapus kolom duration jika kolom tersebut ada di dalam dataset. Komentar pada kode secara eksplisit menyebutkan alasan penghapusan ini: nilai durasi baru dapat diketahui setelah panggilan telepon selesai dilakukan, sehingga tidak relevan atau tidak sesuai untuk skenario model yang bertugas memprediksi keberhasilan sebelum kampanye atau panggilan itu terjadi.
- Rekayasa Fitur 1 (pdays_contacted): Kode membuat fitur baru bernama pdays_contacted yang berfungsi sebagai indikator apakah nasabah pernah dihubungi pada kampanye sebelumnya. Logika ini dibangun dengan mengubah kolom pdays menjadi format numerik, lalu mengevaluasi apakah nilainya tidak sama dengan -1, dan mengonversi hasil logika (True/False) tersebut menjadi tipe data integer (int8).

- Rekayasa Fitur 2 (balance_per_age): Kode membentuk fitur rasio baru bernama balance_per_age dengan cara membagi nilai pada kolom balance (saldo) dengan kolom age (usia). Kode ini juga dilengkapi penanganan error matematis dengan mengubah angka usia 0 dan hasil tak terhingga (np.inf, -np.inf) menjadi Not a Number (np.nan) untuk mencegah kerusakan pada perhitungan matriks selanjutnya.
- Rekayasa Fitur 3 (pdays_recency_group): Kode membentuk fitur kategorikal yang mengelompokkan jarak hari sejak kontak sebelumnya berdasarkan nilai pdays. Kategorinya terdiri dari Never_contacted, Recent_0_30_days, Medium_31_90_days, Long_over_90_days, dan Unknown.
- Rekayasa Fitur 4 (campaign_intensity): Kode membuat fitur kategorikal yang menggambarkan intensitas kontak selama kampanye berjalan berdasarkan nilai campaign. Pengelompokan dilakukan menjadi Low_1_contact untuk satu kali kontak, Medium_2_3_contacts untuk dua hingga tiga kali kontak, dan High_4plus_contacts untuk empat kali kontak atau lebih.
- Verifikasi Hasil: Blok terakhir menggunakan fungsi print untuk memastikan dan menampilkan status apakah duration sudah benar-benar terhapus, menampilkan nama fitur-fitur baru yang berhasil ditambahkan, dan menghitung total jumlah fitur (kolom) yang tersisa sebelum memasuki tahap encoding.

```python
# Salin fitur dan hapus duration.
# duration baru diketahui setelah panggilan dilakukan sehingga tidak sesuai
# dengan skenario prediksi sebelum kampanye/panggilan.
X = X_raw.copy()
if "duration" in X.columns:
    X = X.drop(columns=["duration"])

# Fitur tambahan 1: indikator apakah nasabah pernah dihubungi sebelumnya.
pdays_num = pd.to_numeric(X["pdays"], errors="coerce")
X["pdays_contacted"] = (pdays_num != -1).astype("int8")

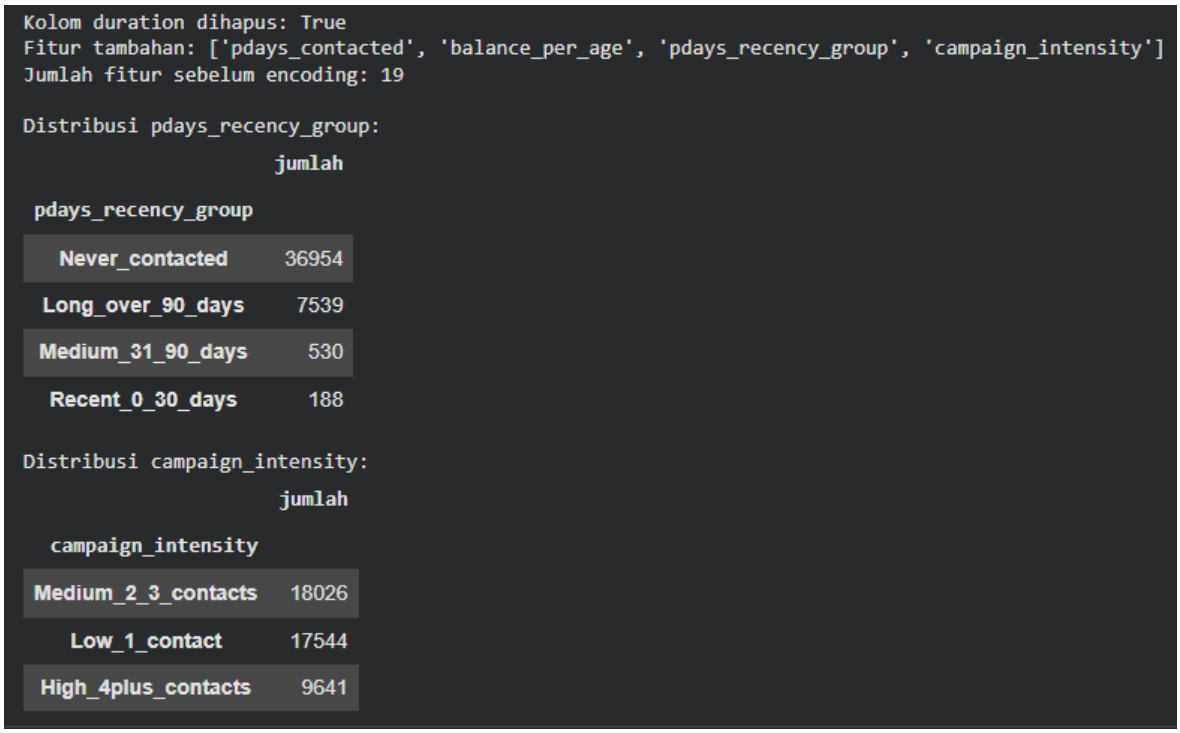
# Fitur tambahan 2: rasio saldo terhadap usia.
X["balance_per_age"] = (
    pd.to_numeric(X["balance"], errors="coerce") /
    pd.to_numeric(X["age"], errors="coerce").replace(0, np.nan)
).replace([np.inf, -np.inf], np.nan)

# Fitur tambahan 3: kelompok jarak hari sejak kontak kampanye sebelumnya.
X["pdays_recency_group"] = np.select(
    [
        pdays_num.eq(-1),
        pdays_num.ge(0) & pdays_num.le(30),
        pdays_num.ge(31) & pdays_num.le(90),
        pdays_num.gt(90)
    ],
    ["Never_contacted", "Recent_0_30_days", "Medium_31_90_days", "Long_over_90_days"],
    default="Unknown"
)

# Fitur tambahan 4: kelompok intensitas kontak pada kampanye saat ini.
campaign_num = pd.to_numeric(X["campaign"], errors="coerce")
X["campaign_intensity"] = np.select(
    [
        campaign_num.eq(1),
        campaign_num.ge(2) & campaign_num.le(3),
        campaign_num.ge(4)
    ],
    ["Low_1_contact", "Medium_2_3_contacts", "High_4plus_contacts"],
    default="Unknown"
)

added_features = [
    "pdays_contacted", "balance_per_age",
    "pdays_recency_group", "campaign_intensity"
]
print("Kolom duration dihapus:", "duration" not in X.columns)
print("Fitur tambahan:", added_features)
print("Jumlah fitur sebelum encoding:", X.shape[1])
print("\nDistribusi pdays_recency_group:")
display(X["pdays_recency_group"].value_counts(dropna=False).to_frame("jumlah"))
print("\nDistribusi campaign_intensity:")
display(X["campaign_intensity"].value_counts(dropna=False).to_frame("jumlah"))
```

Gambar 8 Code Pembersihan Data



```
Kolom duration dihapus: True
Fitur tambahan: ['pdays_contacted', 'balance_per_age', 'pdays_recency_group', 'campaign_intensity']
Jumlah fitur sebelum encoding: 19

Distribusi pdays_recency_group:
                      jumlah
pdays_recency_group
Never_contacted        36954
Long_over_90_days       7539
Medium_31_90_days        530
Recent_0_30_days         188

Distribusi campaign_intensity:
                      jumlah
campaign_intensity
Medium_2_3_contacts    18026
Low_1_contact          17544
High_4plus_contacts     9641
```

Gambar 9 Output Code Pembersihan Data

b. Normalisasi/standardisasi dan encoding

- Pemisahan Tipe Data dan Perhitungan Parameter: Kode secara otomatis mengidentifikasi daftar fitur numerik dan kategorikal berdasarkan tipe datanya di X_train. Nilai pengganti (imputer) berupa median untuk data numerik dan modus untuk data kategorikal dihitung secara eksklusif hanya dari data latih (X_train). Hal ini merupakan standar prosedur yang wajib untuk memastikan model tidak mempelajari distribusi statistik dari data validasi dan data uji.
- Imputasi dan One-Hot Encoding: Fungsi kustom preprocess_features bertugas mengisi nilai-nilai kosong menggunakan median dan modus yang telah dihitung sebelumnya. Fungsi ini kemudian mengubah data kategorikal menjadi representasi biner (One-Hot Encoding) menggunakan pd.get_dummies dengan argumen drop_first=True. Argumen ini penting untuk membuang satu kategori dasar guna menghindari jebakan variabel dummy (dummy variable trap) yang dapat menyebabkan multikolinearitas sempurna.
- Penyelarasan Dimensi (Reindexing): Setelah data latih di-encode, seluruh nama kolom barunya diekstrak ke dalam variabel feature_columns. Ketika fungsi prapemrosesan diterapkan pada X_val dan X_test, fungsi .reindex(..., fill_value=0) dipanggil untuk memaksakan struktur kolom yang identik dengan data latih. Langkah ini mencegah error jika terdapat kategori tertentu (misalnya

jenis pekerjaan langka) yang muncul di data uji tetapi tidak ada di data latih, atau sebaliknya.

- Penskalaan Fitur (Feature Scaling): Fungsi StandardScaler digunakan untuk menormalisasi rentang nilai seluruh fitur numerik [3]. Scaler ini melakukan proses belajar (fit) sekaligus transformasi pada data latih, dan hanya melakukan transformasi pada data validasi dan data uji tanpa mengubah parameter distribusinya. Seluruh dataframe kemudian dikonversi ke tipe data float32, yang merupakan format optimal untuk alokasi memori dan komputasi tensor pada arsitektur TensorFlow/Keras.
- Verifikasi Dimensi (Sanity Check): Hasil cetak pada konsol (output) menunjukkan bahwa jumlah fitur meningkat menjadi 44 kolom setelah proses encoding selesai. Data kini terbagi menjadi 31.647 sampel latih, 6.782 sampel validasi, dan 6.782 sampel uji. Baris kode assert di bagian akhir berhasil dieksekusi tanpa error, mengonfirmasi bahwa dimensi fitur (kolom) di ketiga set data tersebut sudah sama persis.

```
numeric_features = X_train.select_dtypes(include=np.number).columns.tolist()
categorical_features = [c for c in X_train.columns if c not in numeric_features]

medians = X_train[numeric_features].median()
modes = {}
for col in categorical_features:
    mode_values = X_train[col].mode(dropna=True)
    modes[col] = mode_values.iloc[0] if len(mode_values) else "unknown"

def preprocess_features(frame):
    """Imputasi dan one-hot encoding dengan parameter dari training set."""
    data = frame.copy()
    for col in numeric_features:
        data[col] = pd.to_numeric(data[col], errors="coerce").fillna(medians[col])
    for col in categorical_features:
        data[col] = data[col].fillna(modes[col]).astype(str)
    return pd.get_dummies(data, drop_first=True, dtype=float)

X_train_encoded = preprocess_features(X_train)
feature_columns = X_train_encoded.columns.tolist()
X_val_encoded = preprocess_features(X_val).reindex(columns=feature_columns, fill_value=0)
X_test_encoded = preprocess_features(X_test).reindex(columns=feature_columns, fill_value=0)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded).astype("float32")
X_val_scaled = scaler.transform(X_val_encoded).astype("float32")
X_test_scaled = scaler.transform(X_test_encoded).astype("float32")

print("Jumlah fitur setelah encoding:", len(feature_columns))
print("Shape train:", X_train_scaled.shape)
print("Shape validation:", X_val_scaled.shape)
print("Shape test:", X_test_scaled.shape)
assert X_train_scaled.shape[1] == X_val_scaled.shape[1] == X_test_scaled.shape[1]
```

```
Jumlah fitur setelah encoding: 44
Shape train: (31647, 44)
Shape validation: (6782, 44)
Shape test: (6782, 44)
```

Gambar 10 Preprocessing Data

c. Augmentasi data

Augmentasi data tidak diterapkan dalam penelitian ini karena dataset Bank Marketing merupakan data tabular yang terdiri dari fitur numerik dan kategorikal. Berbeda dengan data citra, teks, atau suara, data tabular tidak memiliki transformasi geometris seperti *rotation*, *flipping*, atau *cropping* yang dapat diterapkan secara langsung tanpa mengubah makna data.

Selain itu, teknik oversampling seperti SMOTE (*Synthetic Minority Over-sampling Technique*) juga tidak digunakan dalam penelitian ini. Penanganan ketidakseimbangan kelas dilakukan menggunakan pendekatan Cost-Sensitive Learning melalui Balanced Class Weighting, yang dijelaskan lebih lanjut pada bagian penanganan ketidakseimbangan kelas.

d. Missing value, outlier, dan ketidakseimbangan kelas

- Penanganan Missing Value

  Berdasarkan hasil pemeriksaan missing value pada dataset yang dimuat menggunakan fungsi fetch_ucirepo(id=222), ditemukan beberapa kolom yang memiliki nilai kosong (NaN), yaitu job sebanyak 288 data (0,64%), education sebanyak 1.857 data (4,11%), contact sebanyak 13.020 data (28,80%), dan poutcome sebanyak 36.959 data (81,75%).

  Untuk mengatasi permasalahan tersebut, diterapkan metode imputasi dengan median pada fitur numerik dan modus pada fitur kategorikal. Nilai median dan modus dihitung hanya menggunakan data training (X_train), kemudian digunakan untuk mengisi missing value pada data training, validation, dan testing.

  Nilai kategorikal yang secara eksplisit memiliki kategori unknown tetap dipertahankan sebagai kategori tersendiri. Proses imputasi ini dilakukan untuk memastikan seluruh data dapat diproses oleh model tanpa menyebabkan kebocoran informasi dari data validation maupun testing.

- Keputusan Mengenai Outlier

  Ditemukan nilai ekstrem (outlier) pada beberapa fitur numerik, khususnya balance dan campaign. Nilai-nilai tersebut tidak dihapus maupun dilakukan capping karena masih merupakan bagian dari observasi asli dataset.

Selanjutnya, diterapkan standardisasi menggunakan StandardScaler untuk menyamakan skala antarfitur agar proses pembelajaran jaringan saraf lebih stabil. Namun, standardisasi ini tidak menghilangkan pengaruh outlier sepenuhnya karena StandardScaler tetap sensitif terhadap nilai ekstrem.

- Penanganan Ketidakseimbangan Kelas (*Class Weighting*)

  Nilai bobot masing-masing kelas dihitung secara otomatis menggunakan fungsi compute_class_weight() dari pustaka Scikit-learn dengan metode balanced [4]. Berdasarkan hasil perhitungan, diperoleh bobot kelas 0 (No) sebesar 0,5663 dan kelas 1 (Yes) sebesar 4,2732.

  Bobot tersebut kemudian diterapkan pada setiap sampel data training melalui pembentukan variabel sample_weights. Selanjutnya, fitur, label, dan bobot sampel digabungkan menggunakan tf.data.Dataset.from_tensor_slices() untuk membentuk dataset training berbobot (train_dataset_weighted).

  Dataset tersebut digunakan dalam proses pelatihan model MLP sehingga kesalahan prediksi pada kelas minoritas (Yes) memperoleh kontribusi loss yang lebih besar dibandingkan kelas mayoritas (No). Penerapan ini bertujuan agar model memberikan perhatian lebih terhadap nasabah yang berpotensi berlangganan deposito.

```
classes = np.array([0, 1])
class_weights = compute_class_weight(
    class_weight="balanced", classes=classes, y=y_train
)
class_weight_dict = {
    int(label): float(weight)
    for label, weight in zip(classes, class_weights)
}
print("Class weight:", class_weight_dict)

sample_weights = np.array(
    [class_weight_dict[int(label)] for label in y_train_np],
    dtype="float32"
)
train_dataset_weighted = (
    tf.data.Dataset.from_tensor_slices(
        (X_train_scaled, y_train_np, sample_weights)
    )
    .shuffle(len(X_train_scaled), seed=SEED, reshuffle_each_iteration=True)
    .batch(BATCH_SIZE)
    .prefetch(AUTOTUNE)
)
print("Weighted dataset batches:", len(train_dataset_weighted))
```

```
Class weight: {0: 0.5662575150300602, 1: 4.273156899810964}
Weighted dataset batches: 495
```

Gambar 11 Perhitungan Class Weight

Penerapan Balanced Class Weighting merupakan salah satu pendekatan Cost-Sensitive Learning yang digunakan untuk mengatasi ketidakseimbangan kelas tanpa mengubah jumlah sampel pada dataset. Dengan memberikan bobot lebih besar pada kelas Yes, model diharapkan dapat memberikan perhatian lebih terhadap kesalahan prediksi pada kelas minoritas selama proses pelatihan. Pendekatan ini kemudian digunakan secara konsisten pada eksperimen model MLP Baseline dan MLP Tuned.

Pada pengembangan UTS, penerapan Class Weighting tidak hanya digunakan sebagai pendekatan penanganan ketidakseimbangan kelas, tetapi juga dibandingkan dengan skenario tanpa Class Weighting. Perbandingan ini dilakukan untuk mengetahui apakah pemberian bobot pada kelas minoritas benar-benar memberikan hasil yang lebih baik pada model MLP. Hasil eksperimen menunjukkan bahwa model MLP Tuned tanpa Class Weighting memberikan nilai F1-score validation tertinggi setelah dilakukan Threshold Tuning. Oleh karena itu, model final yang dipilih menggunakan konfigurasi tanpa Class Weighting.

e. Pembagian data dan pencegahan kebocoran

Subset	Proporsi	Jumlah data	Keterangan
Training	70%	31.647	Menggunakan *Stratified Sampling* untuk pembelajaran bobot model.
Validation	15%	6.782	Menggunakan *Stratified Sampling* untuk evaluasi *early stopping* dan penyetelan *hyperparameter*.
Testing	15%	6.782	Menggunakan *Stratified Sampling* untuk pengujian objektif performa akhir model.

Tabel 3 Tabel Pembagian Data

```
Jumlah fitur setelah encoding: 44
Shape train: (31647, 44)
Shape validation: (6782, 44)
Shape test: (6782, 44)
```

Gambar 12 Output Code Pembagian Data

Strategi Pencegahan Kebocoran Data (*Data Leakage Prevention*):

1. Isolasi Pemisahan Subset (Stratified Splitting): Pembagian dataset dilakukan menggunakan fungsi train_test_split berparameter stratify=y dengan rasio 70:15:15. Hal ini menjamin bahwa proporsi ketidakseimbangan kelas (88,3% : 11,7%) terdistribusi secara identik dan presisi di ketiga subset data.

2. Isolasi Standardisasi Fitur: Proses standardisasi menggunakan StandardScaler hanya dilakukan dengan mempelajari parameter statistik dari subset training melalui fungsi fit_transform(). Parameter berupa nilai rata-rata (mean) dan standar deviasi (standard deviation) dari data training tersebut kemudian digunakan untuk mentransformasikan data validation dan testing menggunakan fungsi transform() tanpa menghitung ulang parameter distribusinya. Dengan cara ini, informasi statistik dari data validation dan testing tidak digunakan dalam proses pembentukan parameter standardisasi model.

3. Pengisolasian Data Uji (*Test Set Isolation*): Subset data uji (15% / 6.782 sampel) tidak pernah diikutsertakan dalam iterasi *epoch* pelatihan maupun pembuatan keputusan penyetelan *hyperparameter* (*learning rate*, *dropout rate*, jumlah neuron). Data uji murni hanya dipanggil satu kali pada tahap evaluasi akhir untuk mengukur kemampuan generalisasi model pada sampel baru.

5. Model yang Digunakan

a. Jenis model

Model yang digunakan adalah Deep Neural Network (DNN) dengan arsitektur Multi-Layer Perceptron (MLP) tergeneralisasi untuk menyelesaikan permasalahan klasifikasi biner (*binary classification*), yaitu memprediksi apakah seorang nasabah akan berlangganan deposito berjangka (yes / 1) atau tidak (no / 0).

b. Arsitektur model

Arsitektur Model: MLP_Baseline

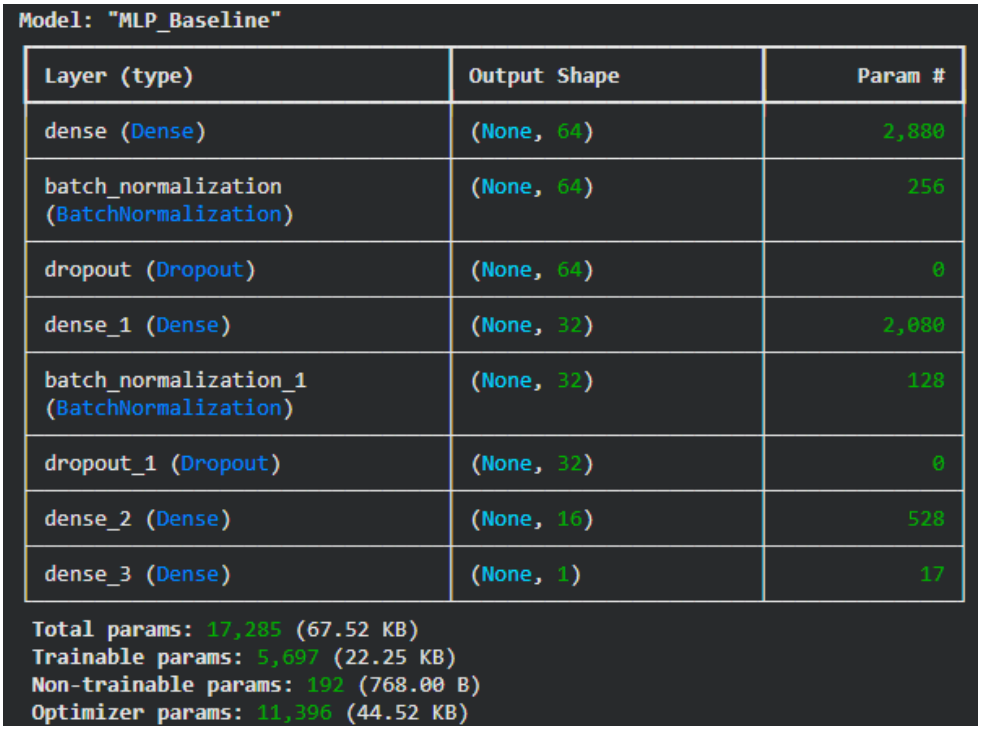
Model ini merupakan rancangan dasar yang dibuat dengan kapasitas (lebar jaringan) yang relatif kecil.

- Struktur Layer: Jaringan ini memiliki empat layer Dense (termasuk output) dengan jumlah neuron yang mengecil secara bertahap, yaitu 64 neuron pada layer pertama, 32

neuron pada layer kedua, 16 neuron pada layer ketiga, dan 1 neuron sebagai output akhir.

- Regulasi: Untuk mencegah overfitting dan mempercepat konvergensi, model ini menggunakan BatchNormalization dan Dropout tepat setelah layer Dense pertama dan kedua.

- Kapasitas Pembelajaran: Karena ukurannya yang ringkas, model ini hanya memiliki Total parameter: 5.889, di mana 5.697 di antaranya adalah parameter yang dapat dilatih (trainable params).



Model: "MLP_Baseline"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,880
batch_normalization (BatchNormalization)	(None, 64)	256
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
batch_normalization_1 (BatchNormalization)	(None, 32)	128
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 17,285 (67.52 KB)
Trainable params: 5,697 (22.25 KB)
Non-trainable params: 192 (768.00 B)
Optimizer params: 11,396 (44.52 KB)

Gambar 13 Arsitektur Model Baseline

Layer	Jenis	Output shape	Parameter	Aktivasi
dense	Dense	(None, 64)	2.880	ReLU
batch_normalization	BatchNormalization	(None, 64)	256	-
dropout	Dropout	(None, 64)	0	-
dense_1	Dense	(None, 32)	2.080	ReLU
batch_normalization_1	BatchNormalization	(None, 32)	128	-

dropout_1	Dropout	(None, 32)	0	-
dense_2	Dense	(None, 16)	528	ReLU
dense_3	Dense	(None, 1)	17	Sigmoid

Tabel 4 Tabel Arsitektur Model Baseline

Model MLP_Tuned (Arsitektur Hasil Tuning)

Model ini merupakan versi yang telah dioptimasi dengan memperbesar kapasitas jaringannya (dibuat lebih lebar) agar mampu mengenali dan mempelajari pola data yang lebih kompleks.

- Struktur Layer: Susunan layernya identik dengan model baseline (terdapat BatchNormalization dan Dropout), namun jumlah neuron pada setiap hidden layer digandakan. Layer pertama memiliki 128 neuron, layer kedua 64 neuron, layer ketiga 32 neuron, dan ditutup dengan 1 neuron output.

- Kapasitas Pembelajaran: Akibat penggandaan jumlah neuron tersebut, ukuran model membengkak secara signifikan dengan Total parameter: 16.897, di mana 16.513 di antaranya adalah parameter yang dapat dilatih.

Model: "MLP_Tuned"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	5,760
batch_normalization (BatchNormalization)	(None, 128)	512
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
batch_normalization_1 (BatchNormalization)	(None, 64)	256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

Total params: 49,925 (195.02 KB)
Trainable params: 16,513 (64.50 KB)
Non-trainable params: 384 (1.50 KB)
Optimizer params: 33,028 (129.02 KB)

Gambar 14 Arsitektur Model Tuned

Layer	Jenis	Output shape	Parameter	Aktivasi
dense	Dense	(None, 128)	5.760	ReLU
batch_normalization	BatchNormalization	(None, 128)	512	-
dropout	Dropout	(None, 128)	0	-
dense_1	Dense	(None, 64)	8.256	ReLU
batch_normalization_1	BatchNormalization	(None, 64)	256	-
dropout_1	Dropout	(None, 64)	0	-
dense_2	Dense	(None, 32)	2.080	ReLU
dense_3	Dense	(None, 1)	33	Sigmoid

Tabel 5 Tabel Arsitektur Model Tuned

Jumlah parameter model pada tabel dihitung berdasarkan total parameter pada masing-masing layer arsitektur, meliputi Dense dan Batch Normalization. Parameter optimizer yang ditampilkan secara terpisah pada output model.summary() tidak dimasukkan ke dalam perhitungan jumlah parameter arsitektur model.

Perbedaan utama dari kedua model ini terletak pada jumlah neuron (lebar jaringan). Model Tuned dirancang jauh lebih kompleks dengan jumlah parameter hampir tiga kali lipat lebih banyak dibandingkan model Baseline. Hal ini biasanya dilakukan dalam tahap eksperimen (seperti pada pengerjaan skripsi Anda) untuk melihat apakah dengan menambahkan neuron, model mampu menangkap fitur klasifikasi secara lebih baik dan meningkatkan performa metrik pada data testing.

c. Konfigurasi pelatihan

Parameter	MLP Baseline	MLP Tuned
Hidden layer	64, 32, 16 neuron	128, 64, 32 neuron
Learning rate	0.001	0.0003
Dropout	0.30, 0.20	0.30, 0.25
L2 Regularization	Tidak digunakan	0.0001
Batch size	64	64

Maksimum epoch	80	100
Optimizer	Adam	Adam
Loss	Binary Crossentropy	Binary Crossentropy
Callback 1	EarlyStopping	EarlyStopping
Monitor EarlyStopping	val_pr_auc	val_pr_auc
Mode	max	max
Patience	8	8
Restore Best Weights	True	True
Callback 2	ReduceLROnPlateau	ReduceLROnPlateau
Monitor ReduceLROnPlateau	val_loss	val_loss
Factor	0,5	0,5
Patience ReduceLR	4	4
Minimum Learning Rate	0,000001	0,000001
Penanganan Imbalance	Balance Class Weighting (Sample Weight)	Balance Class Weighting (Sample Weight)

Tabel 6 Konfigurasi Pelatihan

Pada eksperimen perbandingan model Baseline dan Tuned, kedua arsitektur menggunakan data training yang telah diberikan bobot sampel melalui metode Balanced Class Weighting. Penerapan metode yang sama bertujuan agar perbandingan performa kedua model dilakukan dengan pendekatan penanganan ketidakseimbangan kelas yang konsisten. Dengan demikian, perbedaan performa kedua model terutama dievaluasi berdasarkan perubahan konfigurasi arsitektur dan parameter pelatihan, seperti jumlah neuron, learning rate, dropout, dan penggunaan L2 Regularization pada model Tuned.

Tabel konfigurasi pelatihan menunjukkan konfigurasi awal yang digunakan pada eksperimen model MLP Baseline dan MLP Tuned. Pada pengembangan UTS, eksperimen diperluas dengan membandingkan model MLP Tuned yang menggunakan Class Weighting dan model MLP Tuned tanpa Class Weighting. Perbandingan dilakukan untuk mengetahui pengaruh pembobotan kelas terhadap kemampuan model dalam mengenali kelas minoritas.

Berdasarkan hasil evaluasi pada validation set, model MLP Tuned tanpa Class Weighting dengan threshold 0,17 dipilih sebagai model final karena memperoleh F1-score kelas Yes tertinggi dibandingkan kandidat model lainnya.

d. Alasan pemilihan model dan arsitektur

Model Multilayer Perceptron (MLP) dipilih karena penelitian ini menggunakan dataset Bank Marketing yang memiliki karakteristik data tabular dengan target klasifikasi biner, yaitu memprediksi apakah seorang nasabah akan berlangganan deposito (Yes) atau tidak (No). MLP merupakan salah satu jenis jaringan saraf tiruan yang dapat mempelajari hubungan antara berbagai fitur input dengan hasil klasifikasi. Model ini dipilih karena mampu mengenali pola dan hubungan nonlinier antarfitur yang mungkin memengaruhi keputusan nasabah dalam berlangganan deposito.

Arsitektur MLP yang digunakan terdiri dari beberapa *hidden layer* dengan fungsi aktivasi ReLU dan satu *output layer* dengan fungsi aktivasi Sigmoid. Pada model Baseline, jumlah neuron pada hidden layer secara berurutan adalah 64, 32, dan 16 neuron. Sementara itu, model Tuned menggunakan 128, 64, dan 32 neuron. Penggunaan beberapa hidden layer bertujuan agar model dapat mempelajari pola data secara bertahap, mulai dari pola yang lebih sederhana hingga pola yang lebih kompleks. Fungsi aktivasi ReLU digunakan pada hidden layer, sedangkan Sigmoid digunakan pada output layer untuk menghasilkan probabilitas klasifikasi antara 0 dan 1.

Selain itu, arsitektur model juga dilengkapi dengan beberapa teknik untuk membantu meningkatkan kemampuan generalisasi dan mengurangi risiko *overfitting*. Batch Normalization digunakan untuk membantu menstabilkan proses pembelajaran [5], sedangkan Dropout digunakan untuk mengurangi ketergantungan model terhadap neuron tertentu selama pelatihan [6]. Pada model Tuned, diterapkan pula L2 Regularization dengan nilai 0.0001 untuk membatasi bobot model agar tidak berkembang terlalu besar. Kombinasi teknik tersebut diharapkan dapat membantu model mempelajari pola data dengan lebih baik tanpa terlalu menyesuaikan diri terhadap data pelatihan.

Untuk mengetahui pengaruh konfigurasi arsitektur terhadap performa model, penelitian ini menggunakan dua variasi, yaitu MLP Baseline dan MLP Tuned. Model Baseline digunakan sebagai konfigurasi awal atau pembanding, sedangkan model Tuned menggunakan jumlah neuron yang lebih besar pada setiap hidden layer, learning rate yang

lebih kecil, serta tambahan L2 Regularization. Kedua model menggunakan optimizer Adam dan fungsi loss Binary Crossentropy karena permasalahan yang ditangani merupakan klasifikasi biner. Dengan membandingkan kedua konfigurasi tersebut, dapat diketahui apakah perubahan arsitektur dan pengaturan pelatihan mampu memberikan hasil klasifikasi yang lebih baik [7].

e. Dua fitur unik

Dalam penelitian ini, dilakukan pengembangan fitur (*feature engineering*) dengan menambahkan dua kolom baru pada dataset Bank Marketing. Penambahan fitur ini bertujuan untuk menghasilkan informasi tambahan dari fitur asli yang diharapkan dapat membantu model MLP dalam mengenali pola keberhasilan pemasaran bank. Kedua fitur tambahan tersebut adalah sebagai berikut.

Fitur Unik pada Tugas Proyek 1:

1. **pdays_contacted (Indikator Riwayat Kontak)**

   Fitur pdays_contacted merupakan fitur biner yang menunjukkan apakah seorang nasabah pernah dihubungi dalam kampanye pemasaran sebelumnya.

   - **Cara pembentukan:** Fitur ini dibuat berdasarkan kolom pdays. Jika nilai pdays tidak sama dengan -1, maka pdays_contacted diberi nilai 1. Sebaliknya, jika nilai pdays sama dengan -1, maka diberikan nilai 0.
   - **Alasan penambahan:** Pada dataset asli, nilai -1 pada kolom pdays digunakan sebagai penanda bahwa nasabah belum pernah dihubungi sebelumnya. Dengan menambahkan fitur biner ini, informasi mengenai status pernah atau belum pernah dihubungi dapat dipisahkan dari informasi jarak hari sejak kampanye sebelumnya. Hal ini diharapkan membantu model dalam mengenali pola riwayat kontak nasabah.
   - Bukti dampak:

     Berdasarkan hasil Permutation Importance pada Gambar 15, fitur ini tidak termasuk dalam 15 fitur dengan kontribusi terbesar terhadap performa model. Sementara itu, pada Ablation Study, fitur ini diuji bersamaan dengan balance_per_age. Oleh karena itu, hasil peningkatan PR-AUC dan ROC-AUC pada eksperimen tersebut menunjukkan kontribusi gabungan kedua fitur, bukan kontribusi individual pdays_contacted.

2. **balance_per_age (Rasio Saldo terhadap Usia)**

Fitur balance_per_age merupakan fitur numerik baru yang menggambarkan rasio antara saldo rekening nasabah dengan usia nasabah.

- **Cara pembentukan:** Fitur ini dihitung dengan membagi nilai pada kolom balance dengan nilai pada kolom age, menggunakan rumus:
  balance_per_age = balance / age
  Pada proses pembentukan fitur, nilai usia 0 dan hasil perhitungan yang tidak valid ditangani sebagai missing value.
- **Alasan penambahan:** Nilai saldo rekening secara absolut belum tentu menggambarkan kondisi saldo secara relatif terhadap usia nasabah. Dengan menggunakan rasio saldo terhadap usia, model memperoleh informasi tambahan yang menggabungkan kedua karakteristik tersebut. Fitur ini diharapkan membantu model mengenali hubungan antara usia dan saldo rekening dalam memprediksi keberhasilan pemasaran.
- Bukti dampak:
  Berdasarkan hasil Permutation Importance pada Gambar 15, fitur balance_per_age tidak termasuk dalam 15 fitur dengan kontribusi terbesar terhadap performa model pada eksperimen terbaru. Hal ini menunjukkan bahwa kontribusi fitur tersebut secara individual belum terlihat dominan berdasarkan pengukuran Permutation Importance. Namun, pengaruhnya tetap tercakup dalam pengujian Ablation Study bersama ketiga fitur hasil feature engineering lainnya untuk mengetahui dampak penggunaan seluruh fitur tambahan secara bersamaan.

Fitur Unik pada UTS:

1. **pdays_recency_group (Pengelompokan Jarak Waktu Kontak Sebelumnya)**
   Fitur pdays_recency_group merupakan fitur kategorikal baru yang mengelompokkan jarak waktu sejak nasabah terakhir kali dihubungi dalam kampanye pemasaran sebelumnya.
   - **Cara pembentukan:** Fitur ini dibuat berdasarkan kolom pdays dengan membagi nilai jarak waktu kontak sebelumnya ke dalam beberapa kategori, yaitu:
     - Never_contacted: Nasabah belum pernah dihubungi sebelumnya (nilai pdays = -1).
     - Recent_0_30_days: Nasabah terakhir dihubungi dalam rentang 0–30 hari.
     - Medium_31_90_days: Nasabah terakhir dihubungi dalam rentang 31–90 hari.

- Long_over_90_days: Nasabah terakhir dihubungi lebih dari 90 hari sebelumnya.
- Unknown: Digunakan untuk nilai yang tidak termasuk dalam kategori lainnya.

- **Alasan penambahan:** Nilai pdays pada dataset asli berbentuk numerik sehingga model perlu mengenali hubungan berdasarkan nilai jarak waktu tersebut. Dengan mengelompokkan nilai pdays ke dalam beberapa rentang waktu, informasi mengenai tingkat kebaruan kontak sebelumnya dapat direpresentasikan dalam bentuk kategori. Hal ini diharapkan membantu model dalam mengenali perbedaan pola respons nasabah berdasarkan riwayat waktu kontak.
- Bukti dampak:

  Berdasarkan hasil Permutation Importance pada Gambar 15, fitur pdays_recency_group tidak termasuk dalam 15 fitur dengan kontribusi terbesar terhadap performa model. Namun, kontribusinya tetap tercakup dalam pengujian Ablation Study bersama ketiga fitur hasil feature engineering lainnya.

2. campaign_intensity (Tingkat Intensitas Kontak Kampanye)

Fitur campaign_intensity merupakan fitur kategorikal baru yang menggambarkan tingkat intensitas kontak yang telah dilakukan kepada nasabah dalam kampanye pemasaran saat ini.

- **Cara pembentukan:** Fitur ini dibuat berdasarkan kolom campaign dengan mengelompokkan jumlah kontak yang telah dilakukan kepada nasabah ke dalam beberapa kategori, yaitu:
  - Low_1_contact: Nasabah telah dihubungi sebanyak 1 kali.
  - Medium_2_3_contacts: Nasabah telah dihubungi sebanyak 2–3 kali.
  - High_4plus_contacts: Nasabah telah dihubungi sebanyak 4 kali atau lebih.
  - Unknown: Digunakan untuk nilai yang tidak termasuk dalam kategori lainnya.
- **Alasan penambahan:** Jumlah kontak dalam kampanye pemasaran dapat memberikan gambaran mengenai intensitas pendekatan bank terhadap nasabah. Dengan mengelompokkan jumlah kontak ke dalam beberapa kategori, model diharapkan dapat mengenali pola keberhasilan pemasaran berdasarkan tingkat intensitas komunikasi yang telah dilakukan.
- Bukti dampak:

  Berdasarkan hasil Permutation Importance pada Gambar 15, fitur campaign_intensity tidak termasuk dalam 15 fitur dengan kontribusi terbesar

terhadap performa model. Meskipun demikian, fitur ini tetap digunakan dalam pengujian Ablation Study untuk mengetahui dampak penggunaan seluruh fitur hasil feature engineering secara bersamaan.

	Fitur	Mean F1 decrease	Std
0	month_aug	0.06004	0.00386
1	poutcome_success	0.05338	0.00267
2	day_of_week	0.04512	0.00436
3	month_jun	0.03528	0.00129
4	month_may	0.03498	0.00264
5	housing_yes	0.03316	0.00507
6	month_feb	0.03175	0.00196
7	month_nov	0.02403	0.00202
8	month_jul	0.02218	0.00242
9	month_jan	0.01950	0.00119
10	month_mar	0.01264	0.00158
11	month_oct	0.01219	0.00174
12	month_sep	0.01177	0.00279
13	age	0.00870	0.00230
14	education_tertiary	0.00800	0.00505

Gambar 15 Permutation Importance

	Skenario	Threshold	Accuracy	Precision_yes	Recall_yes	F1_yes	ROC_AUC	PR_AUC
0	Tanpa empat fitur engineered @ 0.50	0.5	0.8953	0.6352	0.2459	0.3545	0.7745	0.4310
1	Dengan empat fitur engineered @ 0.50	0.5	0.8937	0.6241	0.2282	0.3343	0.7729	0.4391

Gambar 16 Ablation Study

Evaluasi Gabungan Keempat Fitur pada Tugas Proyek 1 dan UTS:

Berdasarkan hasil Ablation Study pada Gambar 16, dilakukan perbandingan performa model MLP ketika menggunakan empat fitur hasil feature engineering dan ketika keempat fitur tersebut dihapus. Pengujian dilakukan menggunakan model dan konfigurasi yang sama dengan nilai threshold 0,50 agar perbandingan dapat dilakukan secara konsisten. Evaluasi menggunakan data training dan validation, tanpa melibatkan test set.

Hasil pengujian menunjukkan bahwa model tanpa empat fitur engineered memperoleh Accuracy sebesar 0,8953, Precision Yes sebesar 0,6352, Recall Yes sebesar 0,2459, dan F1-score Yes sebesar 0,3545. Sementara itu, model dengan empat fitur engineered memperoleh Accuracy sebesar 0,8937, Precision Yes sebesar 0,6241, Recall Yes sebesar 0,2282, dan F1-score Yes sebesar 0,3343.

Pada metrik berbasis probabilitas, model tanpa fitur tambahan memperoleh ROC-AUC sebesar 0,7745 dan PR-AUC sebesar 0,4310. Adapun model dengan empat fitur tambahan memperoleh ROC-AUC sebesar 0,7729 dan PR-AUC sebesar 0,4391. Dengan

demikian, penambahan empat fitur engineered memberikan peningkatan pada PR-AUC, tetapi belum meningkatkan metrik Accuracy, Precision, Recall, F1-score, maupun ROC-AUC.

Hasil tersebut menunjukkan bahwa penambahan empat fitur hasil feature engineering memberikan pengaruh yang berbeda terhadap masing-masing metrik evaluasi. Oleh karena itu, efektivitas penambahan fitur tidak dapat dinilai hanya berdasarkan satu metrik, melainkan perlu mempertimbangkan keseluruhan hasil evaluasi.

Perlu diperhatikan bahwa Ablation Study ini menghapus keempat fitur tambahan secara bersamaan. Dengan demikian, hasil pengujian menggambarkan kontribusi gabungan dari pdays_contacted, balance_per_age, pdays_recency_group, dan campaign_intensity, bukan kontribusi masing-masing fitur secara individual.

6. Hasil Evaluasi

a. Metrik evaluasi dan alasannya

Dalam penelitian ini, evaluasi model MLP menggunakan beberapa metrik, yaitu Accuracy, Precision, Recall, F1-score, ROC-AUC, dan PR-AUC. Penggunaan beberapa metrik tersebut bertujuan untuk mengetahui kemampuan model secara menyeluruh dalam mengklasifikasikan nasabah yang berlangganan deposito (Yes) dan tidak berlangganan deposito (No). Pemilihan metrik ini juga mempertimbangkan kondisi dataset Bank Marketing yang memiliki distribusi kelas tidak seimbang, sehingga Accuracy saja tidak cukup untuk menggambarkan performa model.

Berikut merupakan metrik evaluasi yang digunakan beserta alasannya:

Metrik	Fungsi	Alasan Penggunaan
Accuracy	Mengukur persentase keseluruhan prediksi yang benar.	Digunakan untuk mengetahui seberapa banyak data yang berhasil diklasifikasikan dengan tepat oleh model.
Precision	Mengukur ketepatan prediksi pada kelas positif (Yes).	Penting untuk mengetahui seberapa banyak nasabah yang diprediksi berlangganan deposito ternyata benar-

		benar berlangganan.
Recall	Mengukur kemampuan model menemukan seluruh data positif yang sebenarnya.	Digunakan untuk mengetahui seberapa baik model mengenali nasabah yang benar-benar berlangganan deposito.
F1-score	Mengukur keseimbangan antara Precision dan Recall.	Digunakan karena penelitian berfokus pada kelas minoritas (Yes), sehingga diperlukan keseimbangan antara ketepatan prediksi dan kemampuan menemukan kelas positif.
ROC-AUC	Mengukur kemampuan model membedakan kelas positif dan negatif berdasarkan berbagai threshold.	Digunakan untuk mengetahui kemampuan model dalam membedakan nasabah yang berlangganan dan tidak berlangganan deposito.
PR-AUC	Mengukur hubungan antara Precision dan Recall pada berbagai threshold.	Digunakan karena dataset memiliki ketidakseimbangan kelas dan penelitian berfokus pada kemampuan mengenali kelas minoritas (Yes).

Tabel 7 Fungsi dan Alasan Penggunaan Metrik

Alasan utama pemilihan metrik:

Metrik yang menjadi perhatian utama dalam penelitian ini adalah Precision, Recall, dan F1-score pada kelas Yes. Hal ini karena tujuan utama klasifikasi bukan hanya memperoleh akurasi tinggi, tetapi juga mampu mengidentifikasi nasabah yang berpotensi berlangganan deposito. Recall digunakan untuk mengukur kemampuan model menemukan nasabah tersebut, sedangkan Precision digunakan untuk mengukur ketepatan hasil prediksinya. F1-score kemudian digunakan untuk melihat keseimbangan antara kedua metrik tersebut [8].

Selain itu, ROC-AUC dan PR-AUC digunakan sebagai metrik tambahan untuk mengevaluasi kemampuan model berdasarkan berbagai nilai threshold. PR-AUC mendapat perhatian khusus karena lebih berfokus pada hubungan Precision dan Recall untuk kelas

positif, sehingga relevan dengan kondisi dataset yang memiliki jumlah kelas No jauh lebih banyak dibandingkan kelas Yes.

Dengan demikian, penggunaan keenam metrik evaluasi tersebut diharapkan dapat memberikan gambaran yang lebih lengkap mengenai performa model MLP, terutama dalam menangani ketidakseimbangan kelas dan mengenali nasabah yang berpotensi berlangganan deposito.

b. Nilai metrik pada data testing

Evaluasi akhir dilakukan menggunakan data testing sebanyak 6.782 data atau 15% dari keseluruhan dataset. Model yang digunakan adalah MLP Tuned tanpa Class Weighting dengan threshold sebesar 0,17. Model ini dipilih berdasarkan hasil perbandingan pada validation set karena memperoleh nilai F1-score kelas Yes tertinggi dibandingkan kandidat model lainnya.

Metrik Evaluasi	Nilai	Persentase
Accuracy	0.8657	86,57%
Precision (Yes)	0.4312	43,12%
Recall (Yes)	0.4666	46,66%
F1-score (Yes)	0.4482	44,82%
ROC-AUC	0.7713	77,13%
PR-AUC	0.3959	39,59%

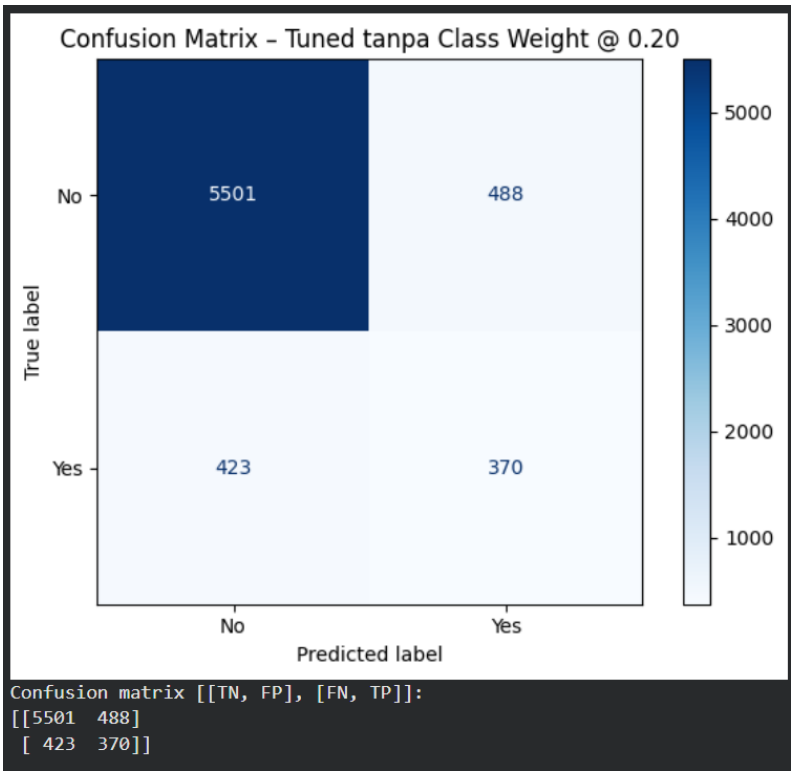
Tabel 8 Hasil Data Testing

Berdasarkan hasil pengujian, model memperoleh Accuracy sebesar 86,57%, yang menunjukkan bahwa sebagian besar data testing berhasil diklasifikasikan dengan benar. Precision kelas Yes sebesar 43,12% menunjukkan bahwa dari seluruh nasabah yang diprediksi berlangganan deposito, sekitar 43,12% merupakan prediksi yang benar.

Recall kelas Yes sebesar 46,66% menunjukkan bahwa model berhasil mengenali sekitar 46,66% dari seluruh nasabah yang sebenarnya berlangganan deposito. Sementara itu, F1-score sebesar 44,82% menunjukkan keseimbangan antara Precision dan Recall pada kelas positif.

Nilai ROC-AUC sebesar 0,7713 menunjukkan kemampuan model dalam membedakan kelas Yes dan No berdasarkan probabilitas prediksi. Sementara itu, PR-AUC sebesar 0,3959 menggambarkan hubungan antara Precision dan Recall pada berbagai nilai threshold.

Secara keseluruhan, model mampu melakukan klasifikasi dengan Accuracy sebesar 86,57%. Namun, kemampuan dalam mengenali kelas minoritas masih memiliki ruang untuk ditingkatkan. Oleh karena itu, hasil prediksi model dapat digunakan sebagai alat bantu untuk menentukan prioritas calon nasabah dalam kegiatan pemasaran, bukan sebagai satu-satunya dasar pengambilan keputusan.



Gambar 17 Confusion Matrix Model MLP Tuned pada Data Testing

Berdasarkan Confusion Matrix, model berhasil mengklasifikasikan 5.501 nasabah kelas No dengan benar (True Negative) dan 370 nasabah kelas Yes dengan benar (True Positive). Namun, masih terdapat 488 nasabah kelas No yang salah diklasifikasikan sebagai Yes (False Positive) dan 423 nasabah kelas Yes yang salah diklasifikasikan sebagai No (False Negative).

Hasil tersebut menunjukkan bahwa model masih menghadapi kesulitan dalam mengidentifikasi sebagian nasabah yang berpotensi berlangganan deposito. Meskipun demikian, model berhasil mengidentifikasi 370 nasabah yang benar-benar berlangganan

deposito pada data testing. Hasil ini juga menunjukkan bahwa penentuan threshold memengaruhi jumlah prediksi positif dan negatif yang dihasilkan model.

c. Grafik loss dan akurasi training vs validation

Grafik Training dan Validation digunakan untuk mengamati perkembangan proses pembelajaran model selama pelatihan. Pada pengembangan UTS, grafik menampilkan perbandingan proses pelatihan tiga skenario model, yaitu MLP Baseline dengan Class Weighting, MLP Tuned dengan Class Weighting, dan MLP Tuned tanpa Class Weighting.

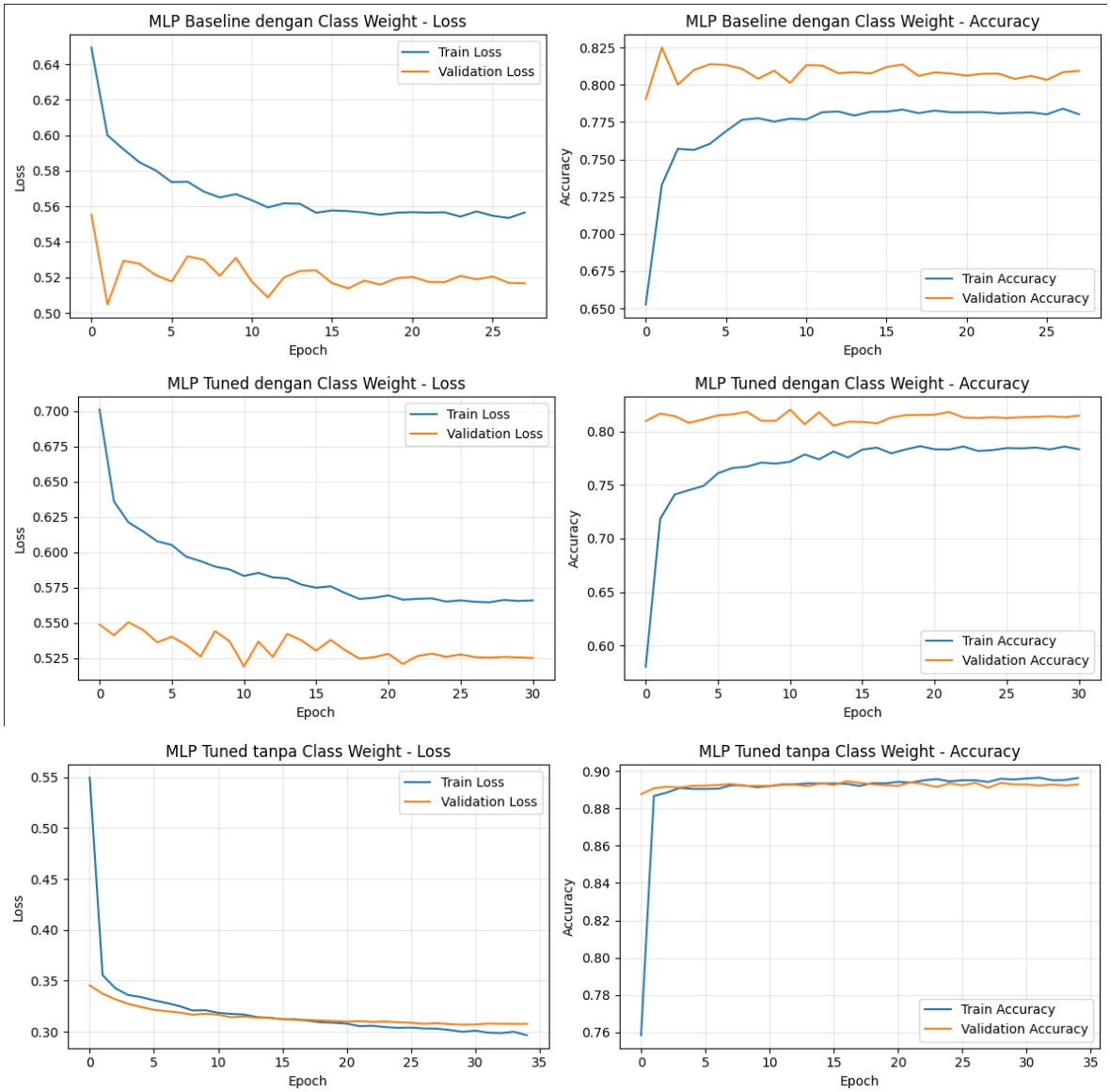
Perbandingan kurva training dan validation digunakan untuk mengamati perubahan nilai loss dan accuracy selama proses pelatihan. Kurva tersebut juga membantu mengidentifikasi kemungkinan terjadinya overfitting atau underfitting. Model yang baik diharapkan mampu mempelajari pola pada data training serta mempertahankan performa yang relatif konsisten pada data validation.

1. Grafik Loss Training vs Validation

Grafik Loss menunjukkan perubahan nilai kesalahan model selama proses pelatihan. Nilai loss yang semakin rendah menunjukkan bahwa kesalahan prediksi model semakin kecil. Perbandingan antara training loss dan validation loss digunakan untuk mengetahui kemampuan model dalam melakukan generalisasi terhadap data yang belum digunakan saat pelatihan.

2. Grafik Accuracy Training vs Validation

Grafik Accuracy menunjukkan persentase prediksi yang benar pada data training dan validation. Semakin tinggi nilai accuracy, semakin banyak prediksi yang sesuai dengan label sebenarnya. Apabila nilai accuracy training dan validation memiliki selisih yang terlalu jauh, hal tersebut dapat menjadi indikasi bahwa model mengalami overfitting.



Gambar 18 Grafik Loss dan Accuracy Curve pada Model Baseline dan Tuned

Gambar 18 menunjukkan kurva pelatihan (loss dan accuracy per epoch) dari tiga skenario model MLP, yaitu MLP Baseline dengan Class Weight, MLP Tuned dengan Class Weight, dan MLP Tuned tanpa Class Weight. Pada model MLP Baseline dengan Class Weight, training loss mengalami penurunan dari awal hingga akhir pelatihan, kemudian mulai melandai. Validation loss juga menunjukkan kecenderungan menurun meskipun terdapat beberapa fluktuasi. Sementara itu, training accuracy meningkat secara bertahap dan validation accuracy berada pada kisaran yang relatif stabil.

Pada model MLP Tuned dengan Class Weight, training loss mengalami penurunan secara bertahap dan validation loss cenderung lebih stabil setelah beberapa epoch awal. Training accuracy meningkat dengan cepat pada awal pelatihan, kemudian bergerak lebih stabil. Validation accuracy juga relatif konsisten sepanjang proses pelatihan.

Selanjutnya, model MLP Tuned tanpa Class Weight menunjukkan penurunan training loss dan validation loss yang cukup konsisten. Kedua kurva loss semakin mendekat dan cenderung stabil pada epoch-epoch akhir. Training accuracy dan validation accuracy juga menunjukkan peningkatan pada awal pelatihan, kemudian berada pada kisaran yang relatif stabil.

Berdasarkan ketiga grafik tersebut, seluruh model mampu mempelajari pola dari data pelatihan. Meskipun terdapat perbedaan nilai loss dan accuracy antar skenario, kurva validasi secara umum menunjukkan pergerakan yang relatif stabil. Tidak terlihat adanya peningkatan validation loss yang tajam secara terus-menerus pada akhir pelatihan. Hal ini menunjukkan bahwa proses pelatihan berjalan dengan cukup baik dan tidak menunjukkan indikasi overfitting yang ekstrem.

d. Perbandingan antarskenario

Pada pengembangan UTS, dilakukan perbandingan terhadap tiga skenario model untuk menentukan konfigurasi yang memberikan performa terbaik dalam mengenali kelas Yes. Ketiga skenario tersebut adalah MLP Baseline dengan Class Weighting, MLP Tuned dengan Class Weighting, serta MLP Tuned tanpa Class Weighting.

Pemilihan model dilakukan menggunakan hasil evaluasi pada validation set. Setiap skenario diuji dengan proses Threshold Tuning untuk memperoleh nilai threshold yang menghasilkan F1-score kelas Yes terbaik.

- **Perbandingan Class Weighting dan Threshold Tuning**

Perbandingan antarskenario dilakukan untuk mengetahui pengaruh penggunaan Class Weighting dan Threshold Tuning terhadap performa model MLP dalam mengklasifikasikan nasabah yang berlangganan deposito (YES). Evaluasi dilakukan dengan memperhatikan Accuracy, Precision, Recall, F1-score, ROC-AUC, dan PR-AUC. Metrik yang menjadi perhatian utama adalah F1-score kelas (YES) karena dapat menunjukkan keseimbangan antara Precision dan Recall pada kelas minoritas.

Skenario	Threshold	Accuracy	Precision	Recall	F1-score	ROC-AUC	PR-AUC
Baseline	0,67	0,8704	0,4470	0,4578	0,4523	0,7724	0,4162

dengan Class Weight							
Tuned dengan Class Weight	0,65	0,8677	0,4385	0,4678	0,4527	0,7729	0,4266
Tuned tanpa Class Weight	0,20	0,8704	0,4496	0,4842	0,4663	0,7729	0,4391

Tabel 9 Perbandingan Metrik Class Weighting dan Threshold Tuning antar Skenario

Berdasarkan hasil tersebut, ketiga skenario menghasilkan performa yang berbeda setelah dilakukan Threshold Tuning. Model Baseline dengan Class Weight menghasilkan F1-score kelas Yes sebesar 0,4523 pada threshold 0,67. Sementara itu, model Tuned dengan Class Weight memperoleh F1-score sebesar 0,4527 pada threshold 0,65.

Adapun model Tuned tanpa Class Weight menghasilkan F1-score tertinggi sebesar 0,4663 pada threshold 0,20. Model ini juga memperoleh nilai PR-AUC sebesar 0,4391, yang merupakan nilai tertinggi di antara ketiga skenario. Nilai ROC-AUC sebesar 0,7729 juga menunjukkan kemampuan diskriminasi model yang kompetitif.

Dengan demikian, berdasarkan kriteria pemilihan model yang mengutamakan F1-score kelas Yes, kemudian Precision, PR-AUC, dan ROC-AUC, model Tuned tanpa Class Weight dipilih sebagai model final. Pemilihan tersebut dilakukan berdasarkan hasil evaluasi pada validation set.

- **Perbandingan Threshold pada Model MLP Tuned**

Selain membandingkan penggunaan Class Weight, dilakukan pengujian threshold pada model MLP Tuned untuk mengetahui pengaruh perubahan nilai ambang batas terhadap hasil klasifikasi.

Skenario	Threshold	Accuracy	Precision	Recall	F1-score	ROC-AUC	PR-AUC
MLP Tuned tanpa Class Weight	0,50	0,8937	0,6241	0,2282	0,3343	0,7729	0,4391
MLP Tuned tanpa Class	0,20	0,8704	0,4496	0,4842	0,4663	0,7729	0,4391

Weight							

Tabel 10 Perbandingan Metrik Threshold Tuning

Berdasarkan tabel tersebut, perubahan threshold dari 0,50 menjadi 0,20 menghasilkan peningkatan Recall dari 0,2282 menjadi 0,4842. Hal ini menunjukkan bahwa model mampu mengenali lebih banyak nasabah yang benar-benar berlangganan deposito.

Namun, peningkatan Recall tersebut diikuti penurunan Accuracy dari 0,8937 menjadi 0,8704 dan Precision dari 0,6241 menjadi 0,4496. Meskipun demikian, F1-score mengalami peningkatan dari 0,3343 menjadi 0,4663.

Nilai ROC-AUC dan PR-AUC tidak mengalami perubahan karena kedua metrik tersebut dihitung berdasarkan probabilitas prediksi model, bukan berdasarkan hasil klasifikasi pada satu threshold tertentu.

Hasil ini menunjukkan bahwa threshold 0,20 memberikan keseimbangan antara Precision dan Recall yang lebih baik dibandingkan threshold default 0,50 berdasarkan F1-score kelas Yes. Oleh karena itu, threshold 0,20 dipilih sebagai threshold final untuk model MLP Tuned tanpa Class Weight berdasarkan hasil evaluasi validation set.

- **Perbandingan Hasil Akhir pada Data Testing**

Setelah model MLP Tuned tanpa Class Weight dengan threshold 0.17 dipilih berdasarkan hasil validation set, model kemudian dievaluasi menggunakan data testing untuk mengetahui kemampuan generalisasinya terhadap data yang belum digunakan dalam proses pelatihan maupun pemilihan threshold.

Metrik	**Hasil Testing**
Accuracy	0,8657
Precision (Yes)	0,4312
Recall (Yes)	0,4666
F1-score (Yes)	0,4482
ROC-AUC	0,7713
PR-AUC	0,3959

Tabel 11 Hasil Akhir Data Testing

Hasil pengujian menunjukkan bahwa model memperoleh Accuracy sebesar 86,57%, Precision kelas Yes sebesar 43,12%, Recall kelas Yes sebesar 46,66%, dan F1-score kelas Yes sebesar 44,82%. Selain itu, model memperoleh ROC-AUC sebesar 0,7713 dan PR-AUC sebesar 0,3959.

Hasil tersebut menunjukkan bahwa model mampu melakukan klasifikasi dengan cukup baik secara keseluruhan. Model berhasil mengenali sekitar 46,66% nasabah yang benar-benar berlangganan deposito. Namun, nilai Precision sebesar 43,12% menunjukkan bahwa masih terdapat prediksi positif yang tidak sesuai dengan kondisi sebenarnya.

Perlu diperhatikan bahwa F1-score sebesar 0,4663 pada Tabel 9 dan Tabel 10 merupakan hasil evaluasi validation set, sedangkan F1-score sebesar 0,4482 merupakan hasil evaluasi test set. Perbedaan tersebut terjadi karena kedua evaluasi menggunakan data yang berbeda. Penurunan nilai F1-score pada test set menunjukkan bahwa performa model pada data yang belum pernah dilihat sedikit lebih rendah dibandingkan hasil validasi.

- **Kesimpulan Perbandingan Antarskenario**

Berdasarkan seluruh hasil eksperimen, dapat disimpulkan bahwa penggunaan Class Weighting dan Threshold Tuning memberikan pengaruh yang berbeda terhadap performa model MLP. Pada threshold default 0,50, penerapan Class Weighting menghasilkan Recall kelas Yes yang lebih tinggi dibandingkan model Tuned tanpa Class Weight. Model Baseline dengan Class Weight memperoleh Recall sebesar 0,5889, sedangkan model Tuned dengan Class Weight memperoleh Recall sebesar 0,5788. Sementara itu, model Tuned tanpa Class Weight memperoleh Recall sebesar 0,2282.

Hasil tersebut menunjukkan bahwa Class Weighting membantu model memberikan perhatian lebih besar terhadap kelas minoritas, sehingga lebih banyak nasabah yang benar-benar berlangganan deposito dapat dikenali. Namun, peningkatan Recall tersebut tidak secara otomatis menghasilkan F1-score yang lebih tinggi pada seluruh skenario.

Sementara itu, Threshold Tuning memberikan fleksibilitas dalam menentukan batas probabilitas klasifikasi agar keseimbangan antara Precision dan Recall menjadi lebih baik. Pada model MLP Tuned tanpa Class Weight, perubahan threshold dari 0,50 menjadi 0,20 meningkatkan F1-score kelas Yes dari 0,3343 menjadi 0,4663.

Berdasarkan perbandingan tiga skenario pada validation set, model MLP Tuned tanpa Class Weight dengan threshold 0,20 memperoleh F1-score tertinggi sebesar 0,4663. Oleh karena itu, model tersebut dipilih sebagai model final untuk dilakukan evaluasi pada data testing.

Pada evaluasi data testing, model final memperoleh Accuracy sebesar 86,57%, Precision kelas Yes sebesar 43,12%, Recall kelas Yes sebesar 46,66%, F1-score kelas Yes sebesar 44,82%, ROC-AUC sebesar 0,7713, dan PR-AUC sebesar 0,3959.

Secara keseluruhan, hasil eksperimen menunjukkan bahwa pemilihan model tidak cukup hanya berdasarkan Accuracy. F1-score kelas Yes, Precision, dan Recall juga perlu diperhatikan karena tujuan utama penelitian adalah mengidentifikasi nasabah yang berpotensi berlangganan deposito, terutama pada kondisi distribusi kelas yang tidak seimbang.

e. Analisis hasil

Berdasarkan hasil eksperimen yang telah dilakukan, model MLP Tuned tanpa Class Weighting dengan threshold 0,20 dipilih sebagai model akhir karena menghasilkan F1-score kelas Yes tertinggi pada validation set, yaitu sebesar 0,4663. Pemilihan model ini dilakukan dengan mempertimbangkan keseimbangan antara Precision dan Recall pada kelas Yes yang merupakan kelas minoritas dalam dataset Bank Marketing.

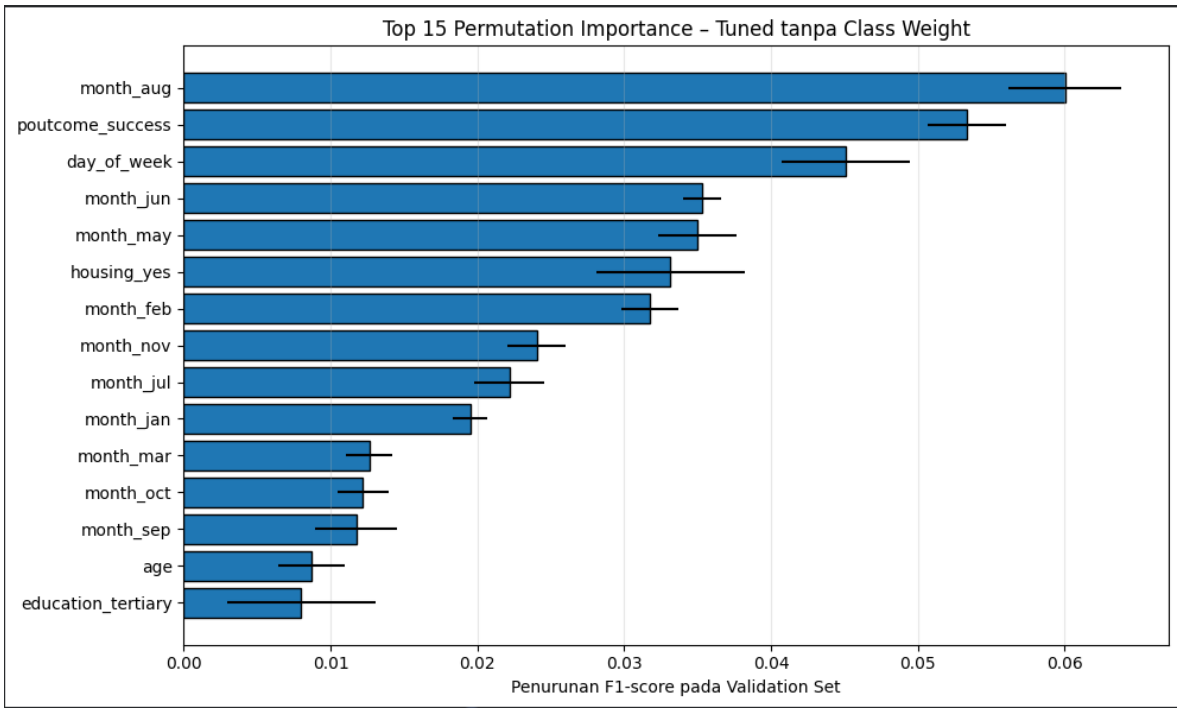
Hasil perbandingan menunjukkan bahwa penerapan Class Weighting tidak selalu menghasilkan peningkatan performa pada seluruh metrik. Pada threshold default 0,50, model Baseline dengan Class Weight memperoleh Recall kelas Yes sebesar 0,5889, sedangkan model Tuned dengan Class Weight memperoleh Recall sebesar 0,5788. Sementara itu, model Tuned tanpa Class Weight memperoleh Recall sebesar 0,2282. Hasil tersebut menunjukkan bahwa Class Weighting membantu model memberikan perhatian lebih besar terhadap kelas minoritas, sehingga lebih banyak nasabah yang benar-benar berlangganan deposito dapat dikenali. Namun, peningkatan Recall tersebut tidak secara otomatis menghasilkan F1-score yang lebih tinggi pada seluruh skenario.

Sementara itu, Threshold Tuning memberikan fleksibilitas dalam menentukan batas probabilitas klasifikasi agar keseimbangan antara Precision dan Recall menjadi lebih baik. Pada model MLP Tuned tanpa Class Weight, perubahan threshold dari 0,50 menjadi 0,20

meningkatkan F1-score kelas Yes dari 0,3343 menjadi 0,4663. Berdasarkan perbandingan ketiga skenario pada validation set, model MLP Tuned tanpa Class Weight dengan threshold 0,20 memperoleh F1-score tertinggi. Oleh karena itu, model tersebut dipilih sebagai model final untuk dilakukan evaluasi pada data testing.

Pada pengujian menggunakan data testing, model final memperoleh Accuracy sebesar 86,57%, Precision kelas Yes sebesar 43,12%, Recall kelas Yes sebesar 46,66%, dan F1-score sebesar 44,82%. Nilai ROC-AUC yang diperoleh adalah 0,7713, sedangkan PR-AUC sebesar 0,3959. Hasil tersebut menunjukkan bahwa model mampu melakukan klasifikasi dengan cukup baik secara keseluruhan, meskipun kemampuan dalam mengenali kelas minoritas masih dapat ditingkatkan.

Berdasarkan Confusion Matrix, terdapat 5.501 prediksi True Negative, 370 True Positive, 488 False Positive, dan 423 False Negative. Hasil tersebut menunjukkan bahwa model masih menghasilkan sejumlah prediksi positif yang tidak tepat serta belum berhasil mengenali seluruh nasabah yang sebenarnya berlangganan deposito. Dengan demikian, hasil prediksi model dapat digunakan sebagai alat bantu dalam menentukan prioritas kampanye pemasaran, bukan sebagai satu-satunya dasar pengambilan keputusan.



Gambar 19 Grafik Permutation Importance pada Model Tuned

Berdasarkan hasil Permutation Importance pada validation set yang ditunjukkan pada Gambar 19, fitur month_aug menunjukkan nilai rata-rata penurunan F1-score

terbesar, yaitu sebesar 0,06004. Selanjutnya, fitur poutcome_success memiliki nilai sebesar 0,05338, diikuti day_of_week sebesar 0,04512, month_jun sebesar 0,03528, dan month_may sebesar 0,03498.

Hasil ini menunjukkan bahwa model memiliki ketergantungan yang relatif lebih besar terhadap beberapa fitur tersebut dalam menghasilkan prediksi. Semakin besar penurunan F1-score setelah suatu fitur diacak, semakin besar kontribusi relatif fitur tersebut terhadap performa prediksi model pada data yang dievaluasi. Namun, hasil Permutation Importance hanya menunjukkan tingkat kontribusi fitur terhadap performa model dan tidak membuktikan hubungan sebab-akibat antara suatu fitur dengan keputusan nasabah [9].

Selain fitur-fitur utama tersebut, penelitian ini juga menggunakan empat fitur hasil rekayasa (feature engineering), yaitu pdays_contacted, balance_per_age, pdays_recency_group, dan campaign_intensity. Berdasarkan hasil Permutation Importance terbaru, keempat fitur tersebut tidak termasuk dalam 15 fitur dengan kontribusi terbesar yang ditampilkan pada Gambar 19. Hal ini menunjukkan bahwa kontribusi masing-masing fitur secara individual belum terlihat dominan berdasarkan pengukuran tersebut. Namun, kondisi ini tidak berarti bahwa fitur-fitur tersebut sama sekali tidak memberikan informasi tambahan bagi model.

Untuk mengetahui dampak penggunaan fitur hasil rekayasa secara keseluruhan, dilakukan Ablation Study dengan membandingkan model yang menggunakan keempat fitur tersebut dan model yang tidak menggunakannya. Hasil pengujian menunjukkan bahwa penggunaan keempat fitur secara bersamaan meningkatkan PR-AUC dari 0,4310 menjadi 0,4391. Namun, F1-score mengalami penurunan dari 0,3545 menjadi 0,3343 dan ROC-AUC menurun dari 0,7745 menjadi 0,7729. Dengan demikian, kontribusi fitur hasil rekayasa perlu dilihat berdasarkan beberapa metrik evaluasi dan tidak hanya berdasarkan satu ukuran performa.

7. Teknologi yang Digunakan

Kategori	Rincian
Bahasa pemrograman	Python Version: 3.13.18
Library dan versi	- NumPy: 2.1.3

Kategori	Rincian
	- Pandas: 2.2.3 - Matplotlib: 3.10.0 - Seaborn: 0.13.2 - Joblib: 1.6.0 - Scikit-Learn: 1.6.1 - TensorFlow: 2.21.0 - Keras: 3.13.2 - ucimlrepo: 0.0.7
Lingkungan pengembangan	Google Colab
Framework aplikasi dan hosting	- Framework Aplikasi (Web Interface): Streamlit (Python) - Platform Hosting / Deployment: Streamlit Community Cloud - Version Control / Integrasi Deployment: GitHub

Tabel 12 Teknologi dan Versi yang digunakan

TensorFlow dengan API Keras digunakan sebagai framework untuk membangun arsitektur Multi-Layer Perceptron, melakukan proses pelatihan, serta menerapkan komponen pendukung seperti Early Stopping, ReduceLROnPlateau, dan metrik AUC dalam proses evaluasi model [10], [11], [12]. Streamlit digunakan untuk membangun antarmuka aplikasi berbasis web yang memungkinkan pengguna memasukkan data nasabah dan memperoleh hasil prediksi dari model yang telah dilatih. Komponen st.form digunakan untuk mengelompokkan input pengguna sebelum data diproses [13].

8. Deployment dan Demo

a. Link deployment

Aplikasi web Bank Deposit Predictor telah berhasil di-deploy menggunakan platform Streamlit Community Cloud sehingga dapat diakses secara online melalui browser tanpa perlu menjalankan aplikasi secara lokal. Aplikasi ini memungkinkan pengguna untuk memasukkan data nasabah dan memperoleh hasil prediksi potensi berlangganan deposito berjangka menggunakan model MLP yang telah dikembangkan.

Link Deployment: https://prediksi-deposito-bank-6nuk6vumea3pydkff6gvrq.streamlit.app/

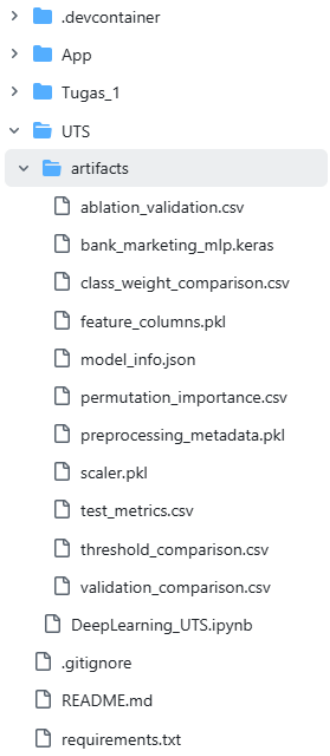
Link Repository GitHub: https://github.com/VincentArfan/Prediksi-Deposito-Bank

Repository GitHub tersebut berisi source code aplikasi, notebook pengembangan model, file artefak hasil pelatihan, serta dokumentasi project melalui README.md. Repository disusun berdasarkan pemisahan folder Tugas 1, UTS, aplikasi, dan laporan untuk memudahkan pengelolaan serta pengembangan project.

b. Tahapan deployment

Berikut adalah penjelasan tahapan deployment untuk aplikasi web Bank Deposit Predictor berbasis Streamlit:

1. Persiapan Model & Dependensi
   - Save/Export Model: Menyimpan model Deep Learning (Multi-Layer Perceptron / MLP) yang telah dilatih beserta scaler dan encoder ke dalam file artefak (seperti format .pkl, .h5, atau .keras).
   - Manajemen Dependensi: Membuat file requirements.txt yang mencantumkan library utama seperti Streamlit, TensorFlow/Keras, pandas, numpy, scikit-learn, dan plotly/matplotlib untuk visualisasi gauge.



Gambar 20 Struktur dan Isi Folder Artifacts

2. Pembuatan Skrip Antarmuka Streamlit (app.py)

- Pemuatan Artefak: Memuat file model dan scaler secara otomatis menggunakan dekorator @st.cache_resource agar proses loading efisien.
- Desain UI: Membangun sidebar untuk konfigurasi model/threshold serta form dua kolom untuk input profil nasabah dan riwayat kampanye.
- Logika Feature Engineering & Prediksi: Membuat fungsi otomatis untuk menghitung fitur baru (seperti balance_per_age dan pengelompokan pdays_recency_group) dari input pengguna sebelum diserahkan ke model untuk prediksi.

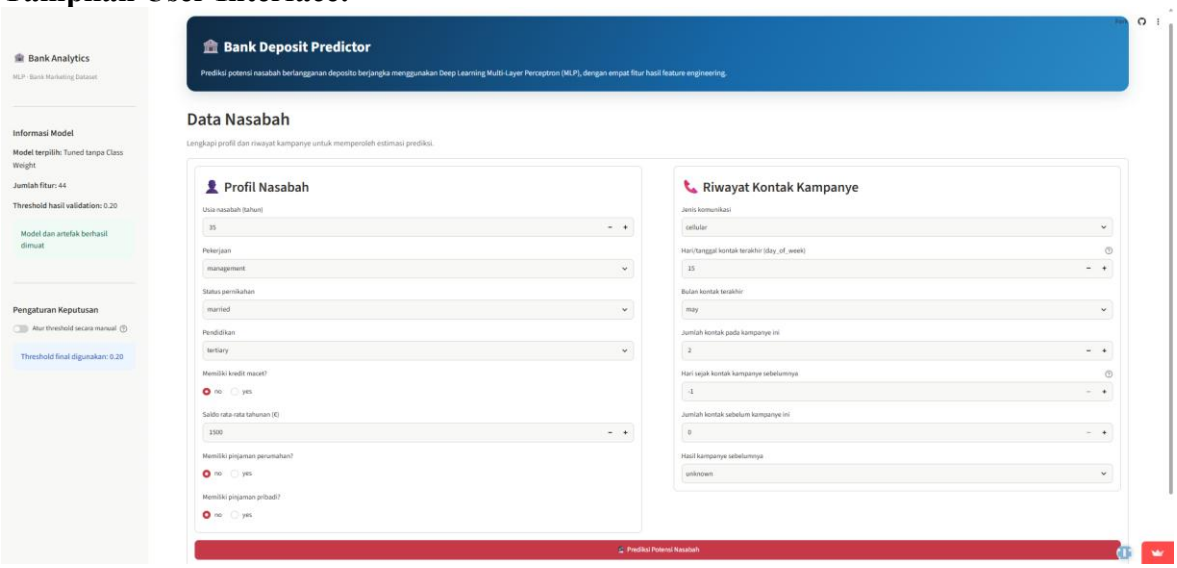
3. Pengujian Lokal (Local Testing)
   - Menjalankan perintah streamlit run app.py di lingkungan lokal (misal: Anaconda/Virtual Environment) untuk memastikan seluruh form, tombol prediksi, kalkulasi threshold, dan gauge chart berjalan tanpa error.

4. Kontrol Versi (Version Control)
   - Memasukkan skrip kode (app_uts.py), file konfigurasi, requirements.txt, dan artefak model ke dalam repositori GitHub.

5. Deployment ke Cloud Platform
   - Streamlit Community Cloud: Menghubungkan repositori GitHub ke akun Streamlit Cloud, memilih repositori dan main branch, lalu melakukan deploy.

c. Penjelasan demo dan deployment

Tampilan User Interface:



Gambar 21 Tampilan Antarmuka Aplikasi Bank Deposit Predictor

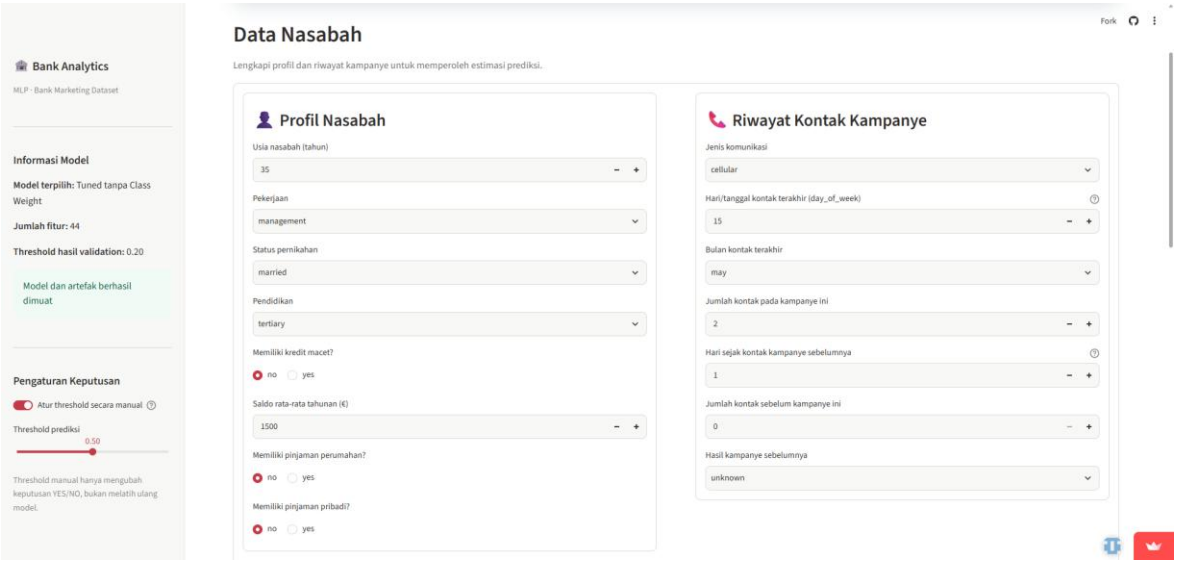
Tampilan antarmuka tersebut merupakan aplikasi web Bank Deposit Predictor berbasis Streamlit yang berfungsi memprediksi potensi nasabah untuk berlangganan deposito berjangka menggunakan model Deep Learning (Multi-Layer Perceptron / MLP).

1. Sidebar Kiri (Pengaturan & Informasi Model)
    - Informasi Model: Menampilkan detail model yang dimuat, yaitu model Tuned tanpa Class Weight, jumlah total fitur yang digenerasi (44 fitur), serta nilai threshold optimal dari hasil validasi (0.20).
    - Pengaturan Keputusan: Menyediakan kontrol manual slider threshold yang bisa diatur untuk pemilihan threshold 0.10 – 0.90 untuk mengubah ambang batas klasifikasi (YES/NO) secara langsung tanpa melatih ulang model.

2. Form Input Data Nasabah
    - Profil Nasabah: Input data demografi dan finansial nasabah, seperti usia (35), pekerjaan (management), status pernikahan (married), pendidikan (tertiary), riwayat kredit macet, saldo rata-rata tahunan (1500), serta kepemilikan pinjaman perumahan atau pribadi.
    - Riwayat Kontak Kampanye: Input detail interaksi kampanye pemasaran, meliputi jenis komunikasi (cellular), hari & bulan kontak terakhir (15 May), jumlah kontak kampanye ini (2), rentang hari sejak kontak sebelumnya (1), jumlah kontak sebelum kampanye ini (0), dan hasil kampanye sebelumnya (unknown).

3. Tombol Prediksi
    - Prediksi Potensi Nasabah: Tombol aksi di bagian bawah untuk memproses seluruh fitur input ke dalam model MLP dan menampilkan output klasifikasi (apakah nasabah berpotensi membuka deposito atau tidak) berdasarkan nilai threshold yang disesuaikan.

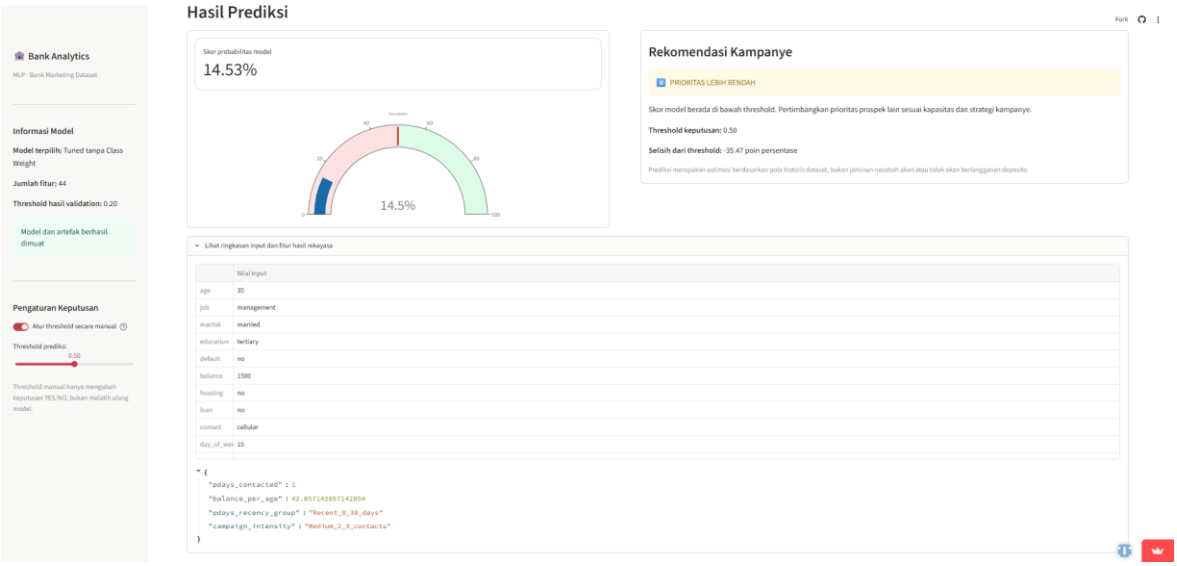
Contoh Input dan Output:



Gambar 22 Contoh Input Aplikasi Bank Deposit Predictor

Pada input aplikasi, kita mencoba untuk mengisi beberapa fitur yang disediakan dengan nilai berikut:

- Usia nasabah : 35
- Pekerjaan : management
- Status pernikahan : married
- Pendidikan : tertiary
- Memiliki kredit macet : no
- Saldo rata-rata tahunan : 1500
- Memiliki pinjaman perumahan : no
- Memiliki pinjaman pribadi : no
- Jenis komunikasi : cellular
- Hari/tanggal kontak terakhir : 15
- Bulan kontak terakhir : may
- Jumlah kontak pada kampanye : 2
- Hari sejak kontak kampanye sebelumnya : 1
- Jumlah kontak sebelum kampanye ini : 0
- Hasil kampanye sebelumnya : unknown



Gambar 23 Output Hasil Prediksi Aplikasi Bank Deposit Predictor

Tampilan antarmuka ini menunjukkan halaman Hasil Prediksi setelah tombol prediksi ditekan pada aplikasi Streamlit.

1. Skor Probabilitas Model & Gauge Chart
   - Skor Probabilitas: Model menghasilkan nilai probabilitas sebesar 14.53%.
   - Visualisasi Gauge: Menampilkan indikator posisi probabilitas (14.5%) yang berada di area merah (rentang nilai rendah).

2. Rekomendasi Kampanye
   - Status: PRIORITAS LEBIH RENDAH.
   - Evaluasi Threshold: Skor model (14.53%) berada di bawah threshold keputusan sebesar 0.50 (50%), dengan selisih -35.47 poin persentase.
   - Saran Sistem: Tim pemasaran disarankan untuk mempertimbangkan prioritas prospek nasabah lain sesuai dengan kapasitas dan strategi kampanye.

3. Ringkasan Input & Fitur Hasil Rekayasa (Feature Engineering)
   - Nilai Input: Menampilkan ringkasan variabel yang dimasukkan sebelumnya (misal: Usia 35, Pekerjaan management, Saldo 1500, Kontak cellular, dll.).
   - Hasil Rekayasa Fitur (JSON): Menampilkan 4 fitur baru yang dibuat oleh sistem:
     - pdays_contacted: 1
     - balance_per_age: 42.857
     - pdays_recency_group: "Recent_0_30_days"
     - campaign_intensity: "Medium_2_3_contacts"

Contoh Pengujian Hasil Prediksi Benar dan Salah

Untuk mengetahui kemampuan model dalam melakukan klasifikasi, dilakukan pengujian menggunakan data nasabah yang memiliki label aktual (actual label). Hasil prediksi model kemudian dibandingkan dengan kondisi sebenarnya untuk mengetahui apakah klasifikasi yang dihasilkan sudah sesuai.

1. **Contoh Prediksi Benar**

   Prediksi dikategorikan benar apabila hasil klasifikasi model sesuai dengan label aktual nasabah. Sebagai contoh, ketika label aktual nasabah adalah Yes dan model juga menghasilkan prediksi Yes, maka hasil tersebut termasuk True Positive (TP). Sementara itu, apabila label aktual adalah No dan model menghasilkan prediksi No, maka hasil tersebut termasuk True Negative (TN).

2. **Contoh Prediksi Salah**

   Prediksi dikategorikan salah apabila hasil klasifikasi model tidak sesuai dengan label aktual nasabah. Kesalahan tersebut dapat berupa False Positive (FP), yaitu ketika model memprediksi Yes tetapi label aktual adalah No, serta False Negative (FN), yaitu ketika model memprediksi No tetapi label aktual adalah Yes.

   Pengujian ini menunjukkan bahwa model MLP masih memiliki kemungkinan menghasilkan kesalahan klasifikasi. Oleh karena itu, hasil prediksi aplikasi sebaiknya digunakan sebagai alat bantu dalam menentukan prioritas kampanye pemasaran dan bukan sebagai satu-satunya dasar pengambilan keputusan.

Demonstrasi Dua Fitur Unik Aplikasi

Aplikasi Bank Deposit Predictor dilengkapi dengan dua fitur utama yang mendukung proses prediksi dan pengambilan keputusan dalam kampanye pemasaran.

3. **Automatic Feature Engineering**

   Fitur ini memungkinkan sistem membentuk empat variabel tambahan secara otomatis berdasarkan data yang dimasukkan pengguna, yaitu pdays_contacted, balance_per_age, pdays_recency_group, dan campaign_intensity.

   Sebagai contoh, berdasarkan input usia 35 tahun dan saldo rekening sebesar 1.500, sistem menghasilkan nilai balance_per_age sebesar 42,857. Selain itu, sistem

mengelompokkan riwayat kontak nasabah dan intensitas kampanye ke dalam kategori yang telah ditentukan. Dengan demikian, pengguna tidak perlu menghitung fitur tambahan tersebut secara manual.

4. **Dynamic Threshold**

   Fitur ini memungkinkan pengguna mengubah nilai threshold melalui slider pada sidebar aplikasi untuk menyesuaikan batas klasifikasi YES dan NO.

   Sebagai contoh, apabila model menghasilkan probabilitas sebesar 14,53%, penggunaan threshold 0,20 akan menghasilkan klasifikasi NO. Namun, ketika threshold diubah menjadi 0,10, hasil klasifikasi berubah menjadi YES. Perubahan tersebut dilakukan tanpa perlu melatih ulang model, sehingga pengguna dapat melihat bagaimana penyesuaian ambang keputusan memengaruhi hasil klasifikasi dan rekomendasi prioritas kampanye.

Penjelasan Singkat Tahapan Deployment

Proses deployment aplikasi dimulai dengan menyimpan model MLP beserta artefak preprocessing yang diperlukan ke dalam folder UTS/artifacts/. Selanjutnya, skrip inferensi dan antarmuka aplikasi dikembangkan menggunakan Streamlit, dengan seluruh dependensi dicatat dalam file requirements.txt.

Source code dan artefak pendukung kemudian diunggah ke repository GitHub. Repository tersebut dihubungkan dengan platform Streamlit Community Cloud dengan menentukan branch main dan file utama aplikasi App/app_uts.py. Setelah proses deployment berhasil, aplikasi dapat diakses secara online melalui browser menggunakan link deployment yang telah dicantumkan pada bagian sebelumnya.

Pengujian dilakukan dengan memasukkan data nasabah, menjalankan proses prediksi, serta memastikan hasil klasifikasi dan fitur tambahan dapat ditampilkan dengan baik pada aplikasi yang telah di-deploy.

9. Lampiran

a. Code

Code Notebook:

- Tahap Instalasi dan Import Library

```
import os
import json
import random
import platform
import warnings
import sys
import importlib.metadata
import matplotlib
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import sklearn
import joblib

from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score, confusion_matrix,
    classification_report, ConfusionMatrixDisplay, roc_curve,
    precision_recall_curve
)
from sklearn.inspection import permutation_importance
from tensorflow import keras
from tensorflow.keras import layers, regularizers


SEED = 42
os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.keras.utils.set_random_seed(SEED)

warnings.filterwarnings("ignore", category=FutureWarning)

print("Python:", platform.python_version())
print("TensorFlow:", tf.__version__)
print("scikit-learn:", sklearn.__version__)
print("pandas:", pd.__version__, "| NumPy:", np.__version__)

print("\n=== Versi Lingkungan & Library ===")
print(f"Python       : {sys.version.split(' ')[0]}")
print(f"NumPy        : {np.__version__}")
print(f"Pandas       : {pd.__version__}")
print(f"Matplotlib   : {matplotlib.__version__}")
print(f"Seaborn      : {sns.__version__}")
print(f"Joblib       : {joblib.__version__}")
print(f"Scikit-Learn : {sklearn.__version__}")
print(f"TensorFlow   : {tf.__version__}")
print(f"Keras        : {tf.keras.__version__}")
print(f"ucimlrepo    : {importlib.metadata.version('ucimlrepo')}")
```

- Tahap Load Dataset UCI Bank Marketing

```
bank = fetch_ucirepo(id=222)
X_raw = bank.data.features.copy()
target_raw = bank.data.targets.iloc[:, 0].copy()

y = (
    target_raw.astype(str)
    .str.strip()
    .str.lower()
    .map({"no": 0, "yes": 1})
)

if y.isna().any():
    raise ValueError("Target mengandung label selain 'yes'/'no'.")

y = y.astype("int32")
df_raw = X_raw.copy()
df_raw["target"] = y

print("Ukuran dataset:", df_raw.shape)
print("Jumlah fitur asli:", X_raw.shape[1])
print("Jumlah duplikat:", df_raw.duplicated().sum())
display(df_raw.head())
display(df_raw.dtypes.rename("dtype").to_frame())
display(df_raw.isna().sum().rename("missing").to_frame())
display(df_raw["target"].value_counts().rename(index={0: "No", 1: "Yes"}).to_frame("jumlah"))
```

- Exploratory Data Analysis (EDA)

```
class_counts = df_raw["target"].value_counts().sort_index()
class_percent = (class_counts / len(df_raw) * 100).round(2)

ax = class_counts.rename(index={0: "No", 1: "Yes"}).plot(
    kind="bar", figsize=(6, 4), rot=0
)
ax.set_title("Distribusi Kelas Target")
ax.set_xlabel("Kelas")
ax.set_ylabel("Jumlah data")
for bar in ax.patches:
    ax.annotate(
        f"{int(bar.get_height()):,}",
        (bar.get_x() + bar.get_width() / 2, bar.get_height()),
        ha="center", va="bottom"
    )
plt.tight_layout()
plt.show()

print("Persentase kelas (%):")
display(class_percent.rename(index={0: "No", 1: "Yes"}).to_frame("persentase"))
```

- Pembersihan data, feature engineering, dan pembagian data

```
X = X_raw.copy()
if "duration" in X.columns:
    X = X.drop(columns=["duration"])

# Fitur tambahan 1: indikator apakah nasabah pernah dihubungi sebelumnya.
pdays_num = pd.to_numeric(X["pdays"], errors="coerce")
X["pdays_contacted"] = (pdays_num != -1).astype("int8")

# Fitur tambahan 2: rasio saldo terhadap usia.
X["balance_per_age"] = (
    pd.to_numeric(X["balance"], errors="coerce") /
    pd.to_numeric(X["age"], errors="coerce").replace(0, np.nan)
).replace([np.inf, -np.inf], np.nan)

# Fitur tambahan 3: kelompok jarak hari sejak kontak kampanye sebelumnya.
X["pdays_recency_group"] = np.select(
    [
        pdays_num.eq(-1),
        pdays_num.ge(0) & pdays_num.le(30),
        pdays_num.ge(31) & pdays_num.le(90),
        pdays_num.gt(90)
    ],
    ["Never_contacted", "Recent_0_30_days", "Medium_31_90_days", "Long_over_90_days"],
    default="Unknown"
)

# Fitur tambahan 4: kelompok intensitas kontak pada kampanye saat ini.
campaign_num = pd.to_numeric(X["campaign"], errors="coerce")
X["campaign_intensity"] = np.select(
    [
        campaign_num.eq(1),
        campaign_num.ge(2) & campaign_num.le(3),
        campaign_num.ge(4)
    ],
    ["Low_1_contact", "Medium_2_3_contacts", "High_4plus_contacts"],
    default="Unknown"
)

added_features = [
    "pdays_contacted", "balance_per_age",
    "pdays_recency_group", "campaign_intensity"
]
print("Kolom duration dihapus:", "duration" not in X.columns)
print("Fitur tambahan:", added_features)
print("Jumlah fitur sebelum encoding:", X.shape[1])
print("\nDistribusi pdays_recency_group:")
display(X["pdays_recency_group"].value_counts(dropna=False).to_frame("jumlah"))
print("\nDistribusi campaign_intensity:")
display(X["campaign_intensity"].value_counts(dropna=False).to_frame("jumlah"))
```

```
# Split stratified 70:15:15
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X, y, test_size=0.15, random_state=SEED, stratify=y
)
X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full,
    test_size=0.1764705882, random_state=SEED,
    stratify=y_train_full
)

split_summary = pd.DataFrame({
    "Subset": ["Train", "Validation", "Test"],
    "Jumlah": [len(y_train), len(y_val), len(y_test)],
    "Persentase": [len(y_train) / len(y), len(y_val) / len(y), len(y_test) / len(y)],
    "Proporsi Yes": [y_train.mean(), y_val.mean(), y_test.mean()]
})
display(split_summary.round(4))
```

- Preprocessing tanpa data leakage

```
numeric_features = X_train.select_dtypes(include=np.number).columns.tolist()
categorical_features = [c for c in X_train.columns if c not in numeric_features]

medians = X_train[numeric_features].median()
modes = {}
for col in categorical_features:
    mode_values = X_train[col].mode(dropna=True)
    modes[col] = mode_values.iloc[0] if len(mode_values) else "unknown"

def preprocess_features(frame):
    """Imputasi dan one-hot encoding dengan parameter dari training set."""
    data = frame.copy()
    for col in numeric_features:
        data[col] = pd.to_numeric(data[col], errors="coerce").fillna(medians[col])
    for col in categorical_features:
        data[col] = data[col].fillna(modes[col]).astype(str)
    return pd.get_dummies(data, drop_first=True, dtype=float)

X_train_encoded = preprocess_features(X_train)
feature_columns = X_train_encoded.columns.tolist()
X_val_encoded = preprocess_features(X_val).reindex(columns=feature_columns, fill_value=0)
X_test_encoded = preprocess_features(X_test).reindex(columns=feature_columns, fill_value=0)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded).astype("float32")
X_val_scaled = scaler.transform(X_val_encoded).astype("float32")
X_test_scaled = scaler.transform(X_test_encoded).astype("float32")

print("Jumlah fitur setelah encoding:", len(feature_columns))
print("Shape train:", X_train_scaled.shape)
print("Shape validation:", X_val_scaled.shape)
print("Shape test:", X_test_scaled.shape)
assert X_train_scaled.shape[1] == X_val_scaled.shape[1] == X_test_scaled.shape[1]
```

- TensorFlow input pipeline menggunakan tf.data

```
BATCH_SIZE = 64
AUTOTUNE = tf.data.AUTOTUNE

y_train_np = y_train.to_numpy(dtype="int32")
y_val_np = y_val.to_numpy(dtype="int32")
y_test_np = y_test.to_numpy(dtype="int32")

train_dataset = (
    tf.data.Dataset.from_tensor_slices((X_train_scaled, y_train_np))
    .shuffle(len(X_train_scaled), seed=SEED, reshuffle_each_iteration=True)
    .batch(BATCH_SIZE)
    .prefetch(AUTOTUNE)
)
val_dataset = (
    tf.data.Dataset.from_tensor_slices((X_val_scaled, y_val_np))
    .batch(BATCH_SIZE)
    .prefetch(AUTOTUNE)
)
test_dataset = (
    tf.data.Dataset.from_tensor_slices((X_test_scaled, y_test_np))
    .batch(BATCH_SIZE)
    .prefetch(AUTOTUNE)
)

print("Train batches:", len(train_dataset))
print("Validation batches:", len(val_dataset))
print("Test batches:", len(test_dataset))
```

- Fungsi Evaluasi dan Metrik

```python
def get_prob(model, X_array):
    return model.predict(X_array, verbose=0).ravel()

def metric_row(y_true, probabilities, threshold=0.5, name="Model"):
    y_pred = (np.asarray(probabilities) >= threshold).astype(int)
    return {
        "Skenario": name,
        "Threshold": float(threshold),
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision_yes": precision_score(y_true, y_pred, zero_division=0),
        "Recall_yes": recall_score(y_true, y_pred, zero_division=0),
        "F1_yes": f1_score(y_true, y_pred, zero_division=0),
        "ROC_AUC": roc_auc_score(y_true, probabilities),
        "PR_AUC": average_precision_score(y_true, probabilities)
    }

THRESHOLD_GRID = np.round(np.arange(0.10, 0.91, 0.01), 2)

def find_best_threshold(y_true, probabilities, model_name):
    rows = [
        metric_row(y_true, probabilities, threshold, model_name)
        for threshold in THRESHOLD_GRID
    ]
    table = pd.DataFrame(rows)
    best = table.sort_values(
        ["F1_yes", "Precision_yes"], ascending=False
    ).iloc[0]
    return float(best["Threshold"]), table, best
```

- Eksperimen Penanganan Ketidakseimbangan Kelas – Class Weighting

```python
classes = np.array([0, 1])
class_weights = compute_class_weight(
    class_weight="balanced", classes=classes, y=y_train
)
class_weight_dict = {
    int(label): float(weight)
    for label, weight in zip(classes, class_weights)
}
print("Class weight:", class_weight_dict)

sample_weights = np.array(
    [class_weight_dict[int(label)] for label in y_train_np],
    dtype="float32"
)
train_dataset_weighted = (
    tf.data.Dataset.from_tensor_slices(
        (X_train_scaled, y_train_np, sample_weights)
    )
    .shuffle(len(X_train_scaled), seed=SEED, reshuffle_each_iteration=True)
    .batch(BATCH_SIZE)
    .prefetch(AUTOTUNE)
)
print("Weighted dataset batches:", len(train_dataset_weighted))
```

- Arsitektur MLP dengan Callback

```python
def build_mlp(input_dim, tuned=False):
    if not tuned:
        model = keras.Sequential([
            layers.Input(shape=(input_dim,)),
            layers.Dense(64, activation="relu"),
            layers.BatchNormalization(),
            layers.Dropout(0.30),
            layers.Dense(32, activation="relu"),
            layers.BatchNormalization(),
            layers.Dropout(0.20),
            layers.Dense(16, activation="relu"),
            layers.Dense(1, activation="sigmoid")
        ], name="MLP_Baseline")
        learning_rate = 1e-3
    else:
        model = keras.Sequential([
            layers.Input(shape=(input_dim,)),
            layers.Dense(128, activation="relu", kernel_regularizer=regularizers.l2(1e-4)),
            layers.BatchNormalization(),
            layers.Dropout(0.30),
            layers.Dense(64, activation="relu", kernel_regularizer=regularizers.l2(1e-4)),
            layers.BatchNormalization(),
            layers.Dropout(0.25),
            layers.Dense(32, activation="relu", kernel_regularizer=regularizers.l2(1e-4)),
            layers.Dense(1, activation="sigmoid")
        ], name="MLP_Tuned")
        learning_rate = 3e-4

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss="binary_crossentropy",
        metrics=[
            keras.metrics.BinaryAccuracy(name="accuracy"),
            keras.metrics.AUC(name="roc_auc"),
            keras.metrics.AUC(name="pr_auc", curve="PR")
        ]
    )
    return model

def make_callbacks():
    return [
        keras.callbacks.EarlyStopping(
            monitor="val_pr_auc", mode="max", patience=8,
            restore_best_weights=True, verbose=1
        ),
        keras.callbacks.ReduceLROnPlateau(
            monitor="val_loss", mode="min", factor=0.5,
            patience=4, min_lr=1e-6, verbose=1
        )
    ]
```

- Eksperimen dampak Class Weighting

```
def train_weighting_experiment(use_class_weight):
    tf.keras.backend.clear_session()
    keras.utils.set_random_seed(SEED)
    model = build_mlp(X_train_scaled.shape[1], tuned=True)
    dataset = train_dataset_weighted if use_class_weight else train_dataset
    history = model.fit(
        dataset,
        validation_data=val_dataset,
        epochs=100,
        callbacks=make_callbacks(),
        verbose=1
    )
    return model, history

model_no_weight, history_no_weight = train_weighting_experiment(False)
model_with_weight, history_with_weight = train_weighting_experiment(True)

prob_no_weight = get_prob(model_no_weight, X_val_scaled)
prob_with_weight = get_prob(model_with_weight, X_val_scaled)

threshold_no_weight, threshold_table_no_weight, best_no_weight = find_best_threshold(
    y_val, prob_no_weight, "Tuned tanpa Class Weight"
)
threshold_with_weight, threshold_table_with_weight, best_with_weight = find_best_threshold(
    y_val, prob_with_weight, "Tuned dengan Class Weight"
)

class_weight_comparison = pd.DataFrame([
    metric_row(y_val, prob_no_weight, 0.50, "Tanpa Class Weight @ 0.50"),
    metric_row(y_val, prob_with_weight, 0.50, "Dengan Class Weight @ 0.50"),
    metric_row(y_val, prob_no_weight, threshold_no_weight,
               f"Tanpa Class Weight @ {threshold_no_weight:.2f}"),
    metric_row(y_val, prob_with_weight, threshold_with_weight,
               f"Dengan Class Weight @ {threshold_with_weight:.2f}")
])
display(class_weight_comparison.round(4))

print("Threshold terbaik tanpa class weight:", threshold_no_weight)
print("Threshold terbaik dengan class weight:", threshold_with_weight)
```

- Training Model Baseline dan Tuned

```
# Baseline
tf.keras.backend.clear_session()
keras.utils.set_random_seed(SEED)
model_baseline = build_mlp(X_train_scaled.shape[1], tuned=False)
history_baseline = model_baseline.fit(
    train_dataset_weighted,
    validation_data=val_dataset,
    epochs=80,
    callbacks=make_callbacks(),
    verbose=1
)

# Tuned
tf.keras.backend.clear_session()
keras.utils.set_random_seed(SEED)
model_tuned = build_mlp(X_train_scaled.shape[1], tuned=True)
history_tuned = model_tuned.fit(
    train_dataset_weighted,
    validation_data=val_dataset,
    epochs=100,
    callbacks=make_callbacks(),
    verbose=1
)

print("Arsitektur Baseline:")
model_baseline.summary()
print("Arsitektur Tuned:")
model_tuned.summary()
```

- Perbandingan model pada Validation Set dan pemilihan Threshold

```
val_base_probs = get_prob(model_baseline, X_val_scaled)
val_tuned_probs = get_prob(model_tuned, X_val_scaled)
val_no_weight_probs = get_prob(model_no_weight, X_val_scaled)

validation_comparison = pd.DataFrame([
    metric_row(y_val, val_base_probs, 0.50, "Baseline dengan Class Weight @ 0.50"),
    metric_row(y_val, val_tuned_probs, 0.50, "Tuned dengan Class Weight @ 0.50"),
    metric_row(y_val, val_no_weight_probs, 0.50, "Tuned tanpa Class Weight @ 0.50")
])
display(validation_comparison.round(4))

baseline_threshold, baseline_threshold_df, baseline_best = find_best_threshold(
    y_val, val_base_probs, "Baseline dengan Class Weight"
)
tuned_threshold, tuned_threshold_df, tuned_best = find_best_threshold(
    y_val, val_tuned_probs, "Tuned dengan Class Weight"
)
no_weight_threshold, no_weight_threshold_df, no_weight_best = find_best_threshold(
    y_val, val_no_weight_probs, "Tuned tanpa Class Weight"
)

threshold_comparison = pd.DataFrame([
    baseline_best,
    tuned_best,
    no_weight_best
])
display(threshold_comparison.round(4))

selected_row = threshold_comparison.sort_values(
    ["F1_yes", "Precision_yes", "PR_AUC", "ROC_AUC"],
    ascending=False
).iloc[0]
```

```
selected_name = str(selected_row["Skenario"])

model_candidates = {
    "Baseline dengan Class Weight": model_baseline,
    "Tuned dengan Class Weight": model_tuned,
    "Tuned tanpa Class Weight": model_no_weight
}
threshold_candidates = {
    "Baseline dengan Class Weight": baseline_threshold,
    "Tuned dengan Class Weight": tuned_threshold,
    "Tuned tanpa Class Weight": no_weight_threshold
}
probability_candidates = {
    "Baseline dengan Class Weight": val_base_probs,
    "Tuned dengan Class Weight": val_tuned_probs,
    "Tuned tanpa Class Weight": val_no_weight_probs
}

selected_model = model_candidates[selected_name]
best_threshold = float(threshold_candidates[selected_name])
selected_val_probs = probability_candidates[selected_name]

selected_architecture = (
    "Baseline" if selected_name.startswith("Baseline") else "Tuned"
)
selected_uses_class_weight = selected_name != "Tuned tanpa Class Weight"

print("Model terpilih:", selected_name)
print("Arsitektur:", selected_architecture)
print("Menggunakan class weighting:", selected_uses_class_weight)
print("Threshold final dari validation:", best_threshold)
```

- Eksperimen Optimasi Threshold – Threshold Tuning

```
plt.figure(figsize=(10, 5))
plt.plot(
    baseline_threshold_df["Threshold"],
    baseline_threshold_df["F1_yes"],
    label="Baseline dengan Class Weight"
)
plt.plot(
    tuned_threshold_df["Threshold"],
    tuned_threshold_df["F1_yes"],
    label="Tuned dengan Class Weight"
)
plt.plot(
    no_weight_threshold_df["Threshold"],
    no_weight_threshold_df["F1_yes"],
    label="Tuned tanpa Class Weight"
)
plt.axvline(
    best_threshold,
    linestyle="--",
    color="black",
    label=f"Threshold terpilih {best_threshold:.2f}"
)
plt.xlabel("Threshold")
plt.ylabel("F1-score kelas Yes")
plt.title("Threshold Tuning pada Validation Set")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

threshold_impact = pd.DataFrame([
    metric_row(y_val, selected_val_probs, 0.50, f"{selected_name} @ 0.50"),
    metric_row(
        y_val, selected_val_probs, best_threshold,
        f"{selected_name} @ {best_threshold:.2f}"
    )
])
display(threshold_impact.round(4))
```

- Evaluasi Final pada test set

```
test_selected_probs = selected_model.predict(test_dataset, verbose=0).ravel()
final_pred = (test_selected_probs >= best_threshold).astype(int)

test_results = pd.DataFrame([
    metric_row(y_test, test_selected_probs, best_threshold,
               f"Final {selected_name} @ {best_threshold:.2f}")
])
display(test_results.round(4))

print("Classification report - model final:")
print(classification_report(
    y_test, final_pred, target_names=["No", "Yes"], zero_division=0
))

cm = confusion_matrix(y_test, final_pred)
ConfusionMatrixDisplay(
    confusion_matrix=cm, display_labels=["No", "Yes"]
).plot(cmap="Blues", values_format="d")
plt.title(f"Confusion Matrix - {selected_name} @ {best_threshold:.2f}")
plt.tight_layout()
plt.show()
print("Confusion matrix [[TN, FP], [FN, TP]]:")
print(cm)
```

- Grafik training dan kurva evaluasi

```python
def plot_history(history, title):
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    # Grafik Loss
    axes[0].plot(history.history["loss"], label="Train Loss")
    axes[0].plot(history.history["val_loss"], label="Validation Loss")
    axes[0].set_title(f"{title} - Loss")
    axes[0].set_xlabel("Epoch")
    axes[0].set_ylabel("Loss")
    axes[0].legend()
    axes[0].grid(alpha=0.3)

    # Grafik Accuracy
    axes[1].plot(history.history["accuracy"], label="Train Accuracy")
    axes[1].plot(history.history["val_accuracy"], label="Validation Accuracy")
    axes[1].set_title(f"{title} - Accuracy")
    axes[1].set_xlabel("Epoch")
    axes[1].set_ylabel("Accuracy")
    axes[1].legend()
    axes[1].grid(alpha=0.3)

    plt.tight_layout()
    plt.show()


plot_history(history_baseline, "MLP Baseline dengan Class Weight")
plot_history(history_tuned, "MLP Tuned dengan Class Weight")
plot_history(history_no_weight, "MLP Tuned tanpa Class Weight")
```

```python
fpr, tpr, _ = roc_curve(y_test, test_selected_probs)
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, label=f"ROC-AUC = {roc_auc_score(y_test, test_selected_probs):.4f}")
plt.plot([0, 1], [0, 1], "--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Final Test")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

precision_curve, recall_curve, _ = precision_recall_curve(y_test, test_selected_probs)
plt.figure(figsize=(6, 5))
plt.plot(recall_curve, precision_curve)
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve - Final Test")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

- Interpretabilitas – Permutation Importance

```python
class KerasProbabilityWrapper:
    """Wrapper sederhana agar model Keras dapat dipakai oleh permutation_importance."""
    def __init__(self, model):
        self.model = model

    def fit(self, X, y=None):
        return self

    def predict(self, X):
        return self.model.predict(X, verbose=0).ravel()

wrapped_model = KerasProbabilityWrapper(selected_model)

def validation_f1_scorer(estimator, X_eval, y_eval):
    probabilities = estimator.predict(X_eval)
    predictions = (probabilities >= best_threshold).astype(int)
    return f1_score(y_eval, predictions, zero_division=0)

importance_result = permutation_importance(
    wrapped_model,
    X_val_scaled,
    y_val,
    scoring=validation_f1_scorer,
    n_repeats=5,
    random_state=SEED
)

importance_order = importance_result.importances_mean.argsort()[::-1]
top_n = min(15, len(feature_columns))
top_idx = importance_order[:top_n]
top_features = [feature_columns[i] for i in top_idx]
top_means = importance_result.importances_mean[top_idx]
top_stds = importance_result.importances_std[top_idx]

plt.figure(figsize=(10, 6))
plt.barh(
    range(top_n), top_means[::-1], xerr=top_stds[::-1],
    align="center", edgecolor="black"
)
plt.yticks(range(top_n), top_features[::-1])
plt.xlabel("Penurunan F1-score pada Validation Set")
plt.title(f"Top {top_n} Permutation Importance - {selected_name}")
plt.grid(axis="x", alpha=0.3)
plt.tight_layout()
plt.show()

permutation_importance_table = pd.DataFrame({
    "Fitur": top_features,
    "Mean F1 decrease": top_means,
    "Std": top_stds
})
display(permutation_importance_table.round(5))
```

- Ablation Study Feature Engineering

```
engineered_features = [
    "pdays_contacted",
    "balance_per_age",
    "pdays_recency_group",
    "campaign_intensity"
]
X_without_engineered = X.drop(columns=engineered_features)

Xab_train_full, _, yab_train_full, _ = train_test_split(
    X_without_engineered, y,
    test_size=0.15, random_state=SEED, stratify=y
)
Xab_train, Xab_val, yab_train, yab_val = train_test_split(
    Xab_train_full, yab_train_full,
    test_size=0.1764705882, random_state=SEED,
    stratify=yab_train_full
)

num_ab = Xab_train.select_dtypes(include=np.number).columns.tolist()
cat_ab = [c for c in Xab_train.columns if c not in num_ab]
med_ab = Xab_train[num_ab].median()
mode_ab = {}
for col in cat_ab:
    vals = Xab_train[col].mode(dropna=True)
    mode_ab[col] = vals.iloc[0] if len(vals) else "unknown"

def preprocess_ablation(frame):
    data = frame.copy()
    for col in num_ab:
        data[col] = pd.to_numeric(data[col], errors="coerce").fillna(med_ab[col])
    for col in cat_ab:
        data[col] = data[col].fillna(mode_ab[col]).astype(str)
    return pd.get_dummies(data, drop_first=True, dtype=float)

Xab_train_enc = preprocess_ablation(Xab_train)
cols_ab = Xab_train_enc.columns.tolist()
Xab_val_enc = preprocess_ablation(Xab_val).reindex(
    columns=cols_ab, fill_value=0
)

scaler_ab = StandardScaler()
Xab_train_scaled = scaler_ab.fit_transform(Xab_train_enc).astype("float32")
```

```
Xab_val_scaled = scaler_ab.transform(Xab_val_enc).astype("float32")

yab_train_np = yab_train.to_numpy(dtype="int32")
yab_val_np = yab_val.to_numpy(dtype="int32")

if selected_uses_class_weight:
    weights_ab = compute_class_weight(
        class_weight="balanced",
        classes=np.array([0, 1]),
        y=yab_train
    )
    weight_map_ab = {
        0: float(weights_ab[0]),
        1: float(weights_ab[1])
    }
    sample_weights_ab = np.array(
        [weight_map_ab[int(label)] for label in yab_train_np],
        dtype="float32"
    )
    train_ab_dataset = (
        tf.data.Dataset.from_tensor_slices(
            (Xab_train_scaled, yab_train_np, sample_weights_ab)
        )
        .shuffle(
            len(Xab_train_scaled),
            seed=SEED,
            reshuffle_each_iteration=True
        )
        .batch(BATCH_SIZE)
        .prefetch(AUTOTUNE)
    )
```

```
else:
    train_ab_dataset = (
        tf.data.Dataset.from_tensor_slices(
            (Xab_train_scaled, yab_train_np)
        )
        .shuffle(
            len(Xab_train_scaled),
            seed=SEED,
            reshuffle_each_iteration=True
        )
        .batch(BATCH_SIZE)
        .prefetch(AUTOTUNE)
    )

val_ab_dataset = (
    tf.data.Dataset.from_tensor_slices((Xab_val_scaled, yab_val_np))
    .batch(BATCH_SIZE)
    .prefetch(AUTOTUNE)
)

tf.keras.backend.clear_session()
keras.utils.set_random_seed(SEED)

model_without_engineered = build_mlp(
    Xab_train_scaled.shape[1],
    tuned=(selected_architecture == "Tuned")
)
history_without_engineered = model_without_engineered.fit(
    train_ab_dataset,
    validation_data=val_ab_dataset,
    epochs=100,
    callbacks=make_callbacks(),
    verbose=1
)

prob_without_engineered = get_prob(
    model_without_engineered,
    Xab_val_scaled
)
```

```
ablation_validation = pd.DataFrame([
    metric_row(
        yab_val,
        prob_without_engineered,
        0.50,
        "Tanpa empat fitur engineered @ 0.50"
    ),
    metric_row(
        y_val,
        selected_val_probs,
        0.50,
        "Dengan empat fitur engineered @ 0.50"
    )
])
display(ablation_validation.round(4))
```

- Kesimpulan otomatis berdasarkan hasil eksekusi

```
final_metrics = test_results.iloc[0]
selected_validation_metrics = threshold_comparison[
    threshold_comparison["Skenario"] == selected_name
].iloc[0]

print(f"Model final terpilih: {selected_name}")
print(f"Arsitektur: {selected_architecture}")
print(f"Class weighting: {selected_uses_class_weight}")
print(f"Threshold terpilih (validation): {best_threshold:.2f}")
print(f"F1 kelas Yes pada validation: {selected_validation_metrics['F1_yes']:.4f}")
print(f"F1 kelas Yes pada test: {final_metrics['F1_yes']:.4f}")
print(f"Precision Yes pada test: {final_metrics['Precision_yes']:.4f}")
print(f"Recall Yes pada test: {final_metrics['Recall_yes']:.4f}")
print(f"Accuracy pada test: {final_metrics['Accuracy']:.4f}")
print(f"ROC-AUC pada test: {final_metrics['ROC_AUC']:.4f}")
print(f"PR-AUC pada test: {final_metrics['PR_AUC']:.4f}")

print("\\nCatatan interpretasi:")
print("Pemilihan model dan threshold dilakukan berdasarkan validation set.")
print("Test set hanya digunakan untuk evaluasi akhir model yang telah dipilih.")
print("Threshold tuning mengubah trade-off precision dan recall, bukan ROC-AUC/PR-AUC.")
print("Class weighting tidak selalu meningkatkan seluruh metrik; interpretasikan hasil setiap skenario.")
```

Code App:

```
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import streamlit as st
import tensorflow as tf

BASE_DIR = Path(__file__).resolve().parent
ARTIFACT_DIR = BASE_DIR.parent / "UTS" / "artifacts"

st.set_page_config(
    page_title="Bank Deposit Predictor",
    page_icon="🏦",
    layout="wide",
    initial_sidebar_state="expanded",
)

st.markdown(
    """
    <style>
    .stApp,
    [data-testid="stAppViewContainer"],
    [data-testid="stMain"] {
        background-color: var(--background-color);
        color: var(--text-color);
    }

    [data-testid="stHeader"] {
        background: transparent;
    }

    .block-container {
        padding-top: 1.6rem;
        padding-bottom: 2.5rem;
    }
```

```
    .hero {
        padding: 1.55rem 1.8rem;
        border-radius: 18px;
        color: #ffffff;
        background: linear-gradient(120deg, #102a43 0%, #1769aa 58%, #2b8ac6 100%);
        box-shadow: 0 10px 28px rgba(16, 42, 67, .14);
        margin-bottom: 1.2rem;
    }
    .hero h1 {
        color: #ffffff !important;
        font-size: 1.85rem;
        margin: 0;
    }
    .hero p {
        color: #e5f2ff !important;
        margin: .45rem 0 0 0;
        font-size: .98rem;
    }

    .section-note {
        color: var(--text-color);
        opacity: .72;
        font-size: .9rem;
        margin-top: -.4rem;
    }

    div[data-testid="stVerticalBlockBorderWrapper"] {
        border-radius: 15px;
        border: 1px solid var(--border-color, rgba(128, 128, 128, .25));
        background-color: var(--secondary-background-color);
        color: var(--text-color);
    }

    [data-testid="stMetric"] {
        background-color: var(--secondary-background-color);
        color: var(--text-color);
        border: 1px solid var(--border-color, rgba(128, 128, 128, .25));
        border-radius: 13px;
        padding: 14px 16px;
    }
    [data-testid="stMain"] p,
    [data-testid="stMain"] label,
    [data-testid="stMain"] small,
    [data-testid="stMain"] .stMarkdown,
    [data-testid="stMain"] [data-testid="stCaptionContainer"] {
        color: var(--text-color);
    }

    div.stButton > button[kind="primary"] {
        border-radius: 10px;
        min-height: 3rem;
        font-weight: 650;
    }

    footer {
        visibility: hidden;
    }
    </style>
    """,
    unsafe_allow_html=True,


st.cache_resource
ef load_artifacts():
    required_files = [
        "bank_marketing_mlp.keras",
        "scaler.pkl",
        "feature_columns.pkl",
        "preprocessing_metadata.pkl",
        "model_info.json",
    ]
    missing = [name for name in required_files if not (ARTIFACT_DIR / name).exists()]
    if missing:
        raise FileNotFoundError(
            "File artefak belum lengkap: " + ", ".join(missing)
        )

    model = tf.keras.models.load_model(
        ARTIFACT_DIR / "bank_marketing_mlp.keras"
    )
```

```
    scaler = joblib.load(ARTIFACT_DIR / "scaler.pkl")
    feature_columns = joblib.load(ARTIFACT_DIR / "feature_columns.pkl")
    metadata = joblib.load(ARTIFACT_DIR / "preprocessing_metadata.pkl")

    with open(ARTIFACT_DIR / "model_info.json", "r", encoding="utf-8") as file:
        model_info = json.load(file)

    return model, scaler, feature_columns, metadata, model_info


try:
    model, scaler, feature_columns, metadata, model_info = load_artifacts()
except Exception as error:
    st.error("Model atau artefak preprocessing tidak dapat dimuat.")
    st.code(str(error))
    st.info(
        "Jalankan notebook sampai bagian penyimpanan artefak. "
        "Pastikan folder artifacts berada satu direktori dengan app.py."
    )
    st.stop()

def prepare_input(raw_input):
    frame = pd.DataFrame([raw_input])

    if "pdays_contacted" in metadata["raw_input_columns"]:
        frame["pdays_contacted"] = (
            pd.to_numeric(frame["pdays"], errors="coerce") != -1
        ).astype("int8")

    if "balance_per_age" in metadata["raw_input_columns"]:
        age_values = pd.to_numeric(frame["age"], errors="coerce").replace(0, np.nan)
        balance_values = pd.to_numeric(frame["balance"], errors="coerce")
        frame["balance_per_age"] = (
            balance_values / age_values
        ).replace([np.inf, -np.inf], np.nan)

    if "pdays_recency_group" in metadata["raw_input_columns"]:
        pdays_value = pd.to_numeric(
            frame["pdays"], errors="coerce"
    ).iloc[0]

    if pd.isna(pdays_value):
        recency_group = "Unknown"
    elif pdays_value == -1:
        recency_group = "Never_contacted"
    elif pdays_value <= 30:
        recency_group = "Recent_0_30_days"
    elif pdays_value <= 90:
        recency_group = "Medium_31_90_days"
    else:
        recency_group = "Long_over_90_days"

    frame["pdays_recency_group"] = recency_group

if "campaign_intensity" in metadata["raw_input_columns"]:
    campaign_value = pd.to_numeric(
        frame["campaign"], errors="coerce"
    ).iloc[0]

    if pd.isna(campaign_value):
        intensity = "Unknown"
    elif campaign_value == 1:
        intensity = "Low_1_contact"
    elif campaign_value <= 3:
        intensity = "Medium_2_3_contacts"
    else:
        intensity = "High_4plus_contacts"

    frame["campaign_intensity"] = intensity

frame = frame.reindex(columns=metadata["raw_input_columns"])

for col in metadata["numeric_features"]:
    frame[col] = pd.to_numeric(
        frame[col], errors="coerce"
    ).fillna(metadata["medians"][col])
```

```
        for col in metadata["categorical_features"]:
            frame[col] = frame[col].fillna(
                metadata["modes"][col]
            ).astype(str)

        encoded = pd.get_dummies(frame, drop_first=True, dtype=float)
        encoded = encoded.reindex(columns=feature_columns, fill_value=0)

        scaled = scaler.transform(encoded).astype("float32")
        return scaled

    st.markdown(
        """
        <div class="hero">
            <h1>🏦 Bank Deposit Predictor</h1>
            <p>Prediksi potensi nasabah berlangganan deposito berjangka
            menggunakan Deep Learning Multi-Layer Perceptron (MLP), dengan empat fitur hasil feature engineering.</p>
        </div>
        """,
        unsafe_allow_html=True,
    )

    official_threshold = float(
        model_info.get("threshold", metadata.get("threshold", 0.20))
    )

    with st.sidebar:
        st.markdown("## 🏦 Bank Analytics")
        st.caption("MLP · Bank Marketing Dataset")
        st.divider()

        st.markdown("### Informasi Model")
        st.write(f"**Model terpilih:** {model_info.get('selected_model', metadata.get('selected_model', 'MLP'))}")
        st.write(f"**Jumlah fitur:** {len(feature_columns)}")
        st.write(f"**Threshold hasil validation:** {official_threshold:.2f}")
        st.success("Model dan artefak berhasil dimuat")
      st.divider()
      st.markdown("### Pengaturan Keputusan")
      use_custom_threshold = st.toggle(
          "Atur threshold secara manual",
          value=False,
          help="Jika dimatikan, aplikasi memakai threshold final hasil validation."
      )

      if use_custom_threshold:
          threshold = st.slider(
              "Threshold prediksi",
              min_value=0.10,
              max_value=0.90,
              value=official_threshold,
              step=0.01,
          )
          st.caption(
              "Threshold manual hanya mengubah keputusan YES/NO, "
              "bukan melatih ulang model."
          )
      else:
          threshold = official_threshold
          st.info(f"Threshold final digunakan: {threshold:.2f}")

st.markdown("## Data Nasabah")
st.markdown(
    '<p class="section-note">Lengkapi profil dan riwayat kampanye untuk memperoleh estimasi prediksi.</p>',
    unsafe_allow_html=True,
)

with st.form("prediction_form"):
    profile_col, campaign_col = st.columns(2, gap="large")

    with profile_col:
        with st.container(border=True):
            st.markdown("### 👤 Profil Nasabah")
            age = st.number_input(
                "Usia nasabah (tahun)", min_value=18, max_value=100, value=35
            )
```

```
        job = st.selectbox(
            "Pekerjaan",
            [
                "admin.", "blue-collar", "entrepreneur", "housemaid",
                "management", "retired", "self-employed", "services",
                "student", "technician", "unemployed", "unknown",
            ],
            index=4,
        )
        marital = st.selectbox(
            "Status pernikahan",
            ["divorced", "married", "single"],
            index=1,
        )
        education = st.selectbox(
            "Pendidikan",
            ["primary", "secondary", "tertiary", "unknown"],
            index=2,
        )
        default = st.radio(
            "Memiliki kredit macet?",
            ["no", "yes"],
            horizontal=True,
        )
        balance = st.number_input(
            "Saldo rata-rata tahunan (€)",
            min_value=-100000,
            max_value=1000000,
            value=1500,
            step=100,
        )
        housing = st.radio(
            "Memiliki pinjaman perumahan?",
            ["no", "yes"],
            horizontal=True,
        )
        loan = st.radio(
            "Memiliki pinjaman pribadi?",
            ["no", "yes"],
            horizontal=True,
        )

with campaign_col:
    with st.container(border=True):
        st.markdown("### 📞 Riwayat Kontak Kampanye")
        contact = st.selectbox(
            "Jenis komunikasi",
            ["cellular", "telephone", "unknown"],
        )
        day_of_week = st.number_input(
            "Hari/tanggal kontak terakhir (day_of_week)",
            min_value=1,
            max_value=31,
            value=15,
            help="Gunakan nilai numerik sesuai kolom day_of_week pada dataset UCI."
        )
        month = st.selectbox(
            "Bulan kontak terakhir",
            [
                "jan", "feb", "mar", "apr", "may", "jun",
                "jul", "aug", "sep", "oct", "nov", "dec",
            ],
            index=4,
        )
        campaign = st.number_input(
            "Jumlah kontak pada kampanye ini",
            min_value=1,
            max_value=100,
            value=2,
        )
        pdays = st.number_input(
            "Hari sejak kontak kampanye sebelumnya",
            min_value=-1,
            max_value=999,
            value=-1,
            help="-1 berarti nasabah belum pernah dihubungi pada kampanye sebelumnya."
        )
```

```
            previous = st.number_input(
                "Jumlah kontak sebelum kampanye ini",
                min_value=0,
                max_value=100,
                value=0,
            )
            poutcome = st.selectbox(
                "Hasil kampanye sebelumnya",
                ["failure", "other", "success", "unknown"],
                index=3,
            )

    submitted = st.form_submit_button(
        "🔮 Prediksi Potensi Nasabah",
        type="primary",
        use_container_width=True,
    )

if submitted:
    raw_input = {
        "age": age,
        "job": job,
        "marital": marital,
        "education": education,
        "default": default,
        "balance": balance,
        "housing": housing,
        "loan": loan,
        "contact": contact,
        "day_of_week": day_of_week,
        "month": month,
        "campaign": campaign,
        "pdays": pdays,
        "previous": previous,
        "poutcome": poutcome,
    }

    try:
        model_input = prepare_input(raw_input)
        score = float(model.predict(model_input, verbose=0).ravel()[0])
        prediction_yes = score >= threshold

        st.divider()
        st.markdown("## Hasil Prediksi")

        score_col, decision_col = st.columns([1, 1.15], gap="large")

        with score_col:
            with st.container(border=True):
                st.metric(
                    "Skor probabilitas model",
                    f"{score * 100:.2f}%"
                )
                gauge = go.Figure(
                    go.Indicator(
                        mode="gauge+number",
                        value=score * 100,
                        number={"suffix": "%", "font": {"size": 28}},
                        title={"text": "Skor prediksi"},
                        gauge={
                            "axis": {"range": [0, 100]},
                            "bar": {"color": "#1769aa"},
                            "steps": [
                                {"range": [0, threshold * 100], "color": "#fee2e2"},
                                {"range": [threshold * 100, 100], "color": "#dcfce7"},
                            ],
                            "threshold": {
                                "line": {"color": "#b91c1c", "width": 3},
                                "thickness": 0.8,
                                "value": threshold * 100,
                            },
                        },
                    )
                )
                gauge.update_layout(
                    height=250,
```

```
                margin=dict(l=18, r=18, t=55, b=12),
                paper_bgcolor="rgba(0,0,0,0)",
            )
            st.plotly_chart(gauge, use_container_width=True)

    with decision_col:
        with st.container(border=True):
            st.markdown("### Rekomendasi Kampanye")
            if prediction_yes:
                st.success("🎯 POTENSIAL – TARGETKAN NASABAH")
                st.markdown(
                    "Skor model berada pada atau di atas threshold. "
                    "Nasabah dapat diprioritaskan untuk tindak lanjut "
                    "kampanye deposito."
                )
            else:
                st.warning("⏸️ PRIORITAS LEBIH RENDAH")
                st.markdown(
                    "Skor model berada di bawah threshold. "
                    "Pertimbangkan prioritas prospek lain sesuai kapasitas "
                    "dan strategi kampanye."
                )

            st.write(f"**Threshold keputusan:** {threshold:.2f}")
            st.write(
                f"**Selisih dari threshold:** "
                f"{(score - threshold) * 100:+.2f} poin persentase"
            )

            st.caption(
                "Prediksi merupakan estimasi berdasarkan pola historis "
                "dataset, bukan jaminan nasabah akan atau tidak akan "
                "berlangganan deposito."
            )

    with st.expander("Lihat ringkasan input dan fitur hasil rekayasa"):
        st.dataframe(
            pd.DataFrame([raw_input]).T.rename(columns={0: "Nilai input"}),
            use_container_width=True,
        )
            if pdays == -1:
                recency_group_display = "Never_contacted"
            elif pdays <= 30:
                recency_group_display = "Recent_0_30_days"
            elif pdays <= 90:
                recency_group_display = "Medium_31_90_days"
            else:
                recency_group_display = "Long_over_90_days"

            if campaign == 1:
                intensity_display = "Low_1_contact"
            elif campaign <= 3:
                intensity_display = "Medium_2_3_contacts"
            else:
                intensity_display = "High_4plus_contacts"

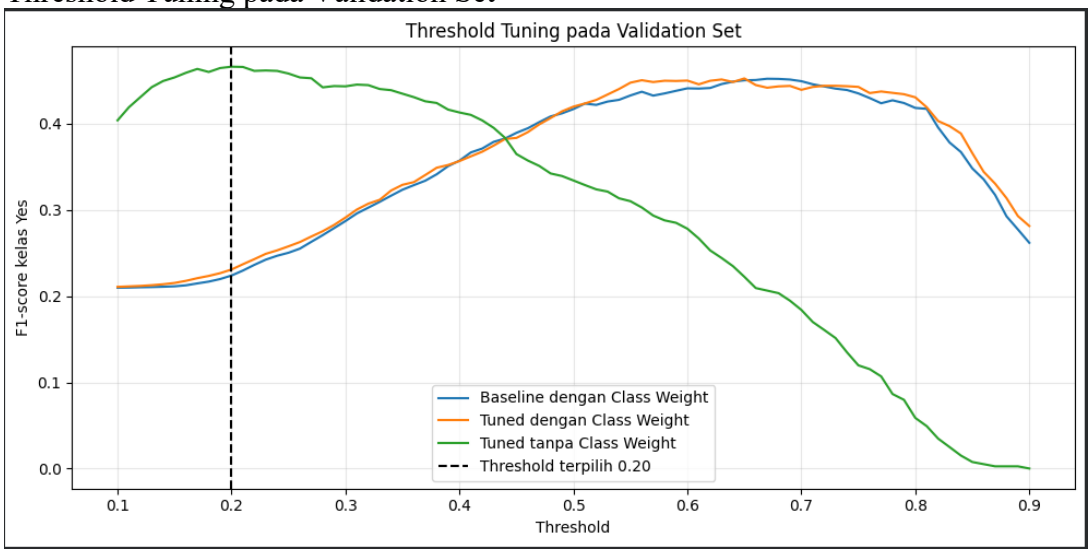
            st.write({
                "pdays_contacted": int(pdays != -1),
                "balance_per_age": float(balance / age) if age else None,
                "pdays_recency_group": recency_group_display,
                "campaign_intensity": intensity_display,
            })

    except Exception as error:
        st.error("Prediksi gagal diproses. Periksa input dan artefak model.")
        st.exception(error)

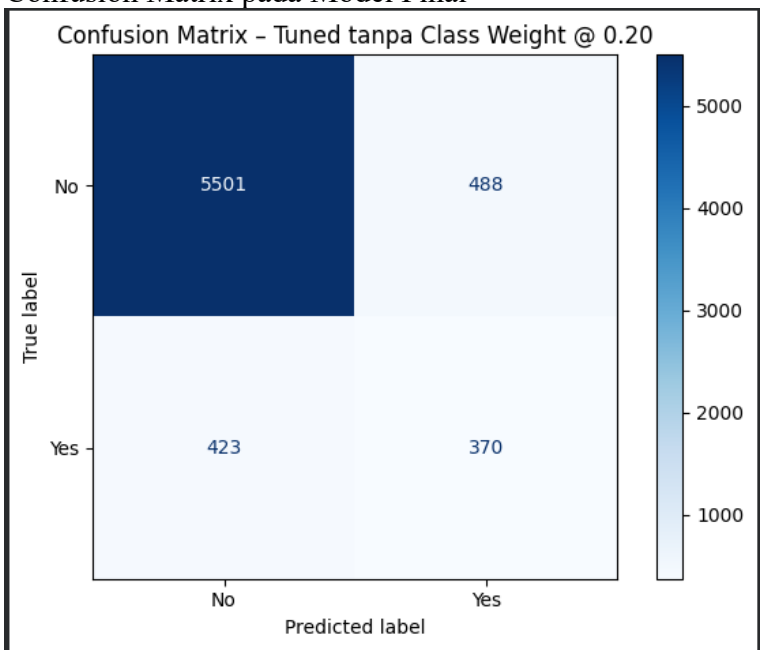
st.divider()
footer_left, footer_right = st.columns(2)
with footer_left:
    st.caption("Deep Learning Project · Bank Marketing UCI Dataset")
with footer_right:
    st.caption("Multi-Layer Perceptron (MLP) · TensorFlow · Streamlit")
```

b. Grafik Evaluasi

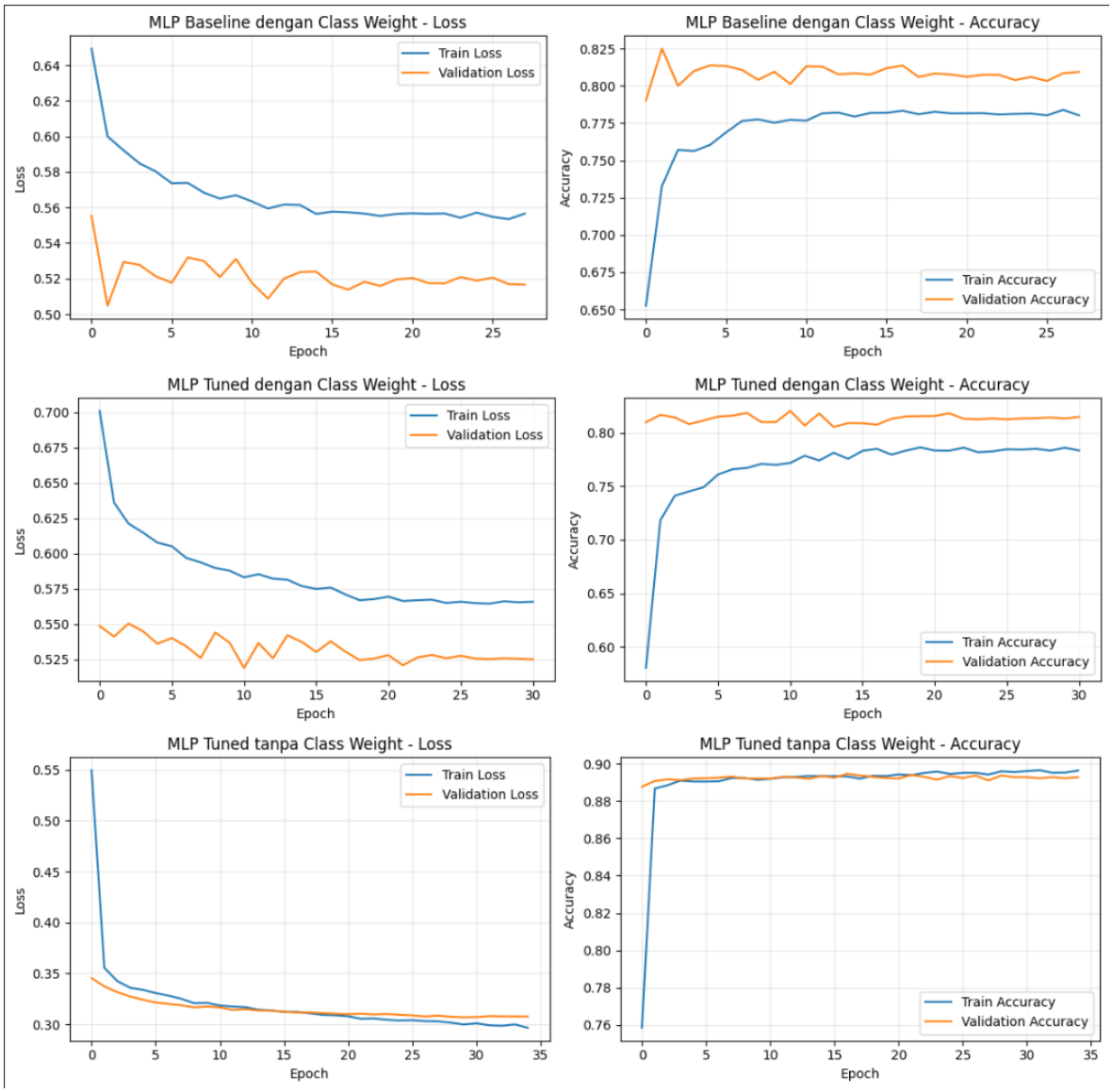
- Threshold Tuning pada Validation Set



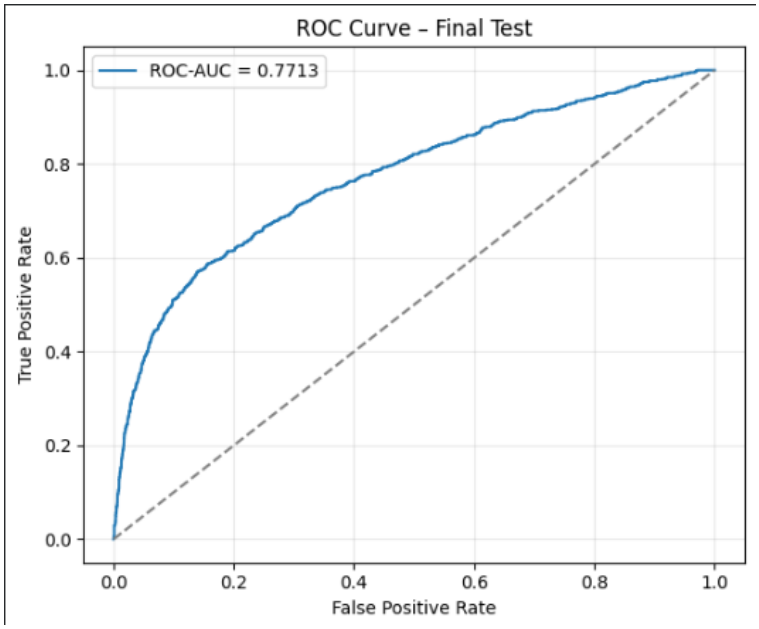
- Confusion Matrix pada Model Final

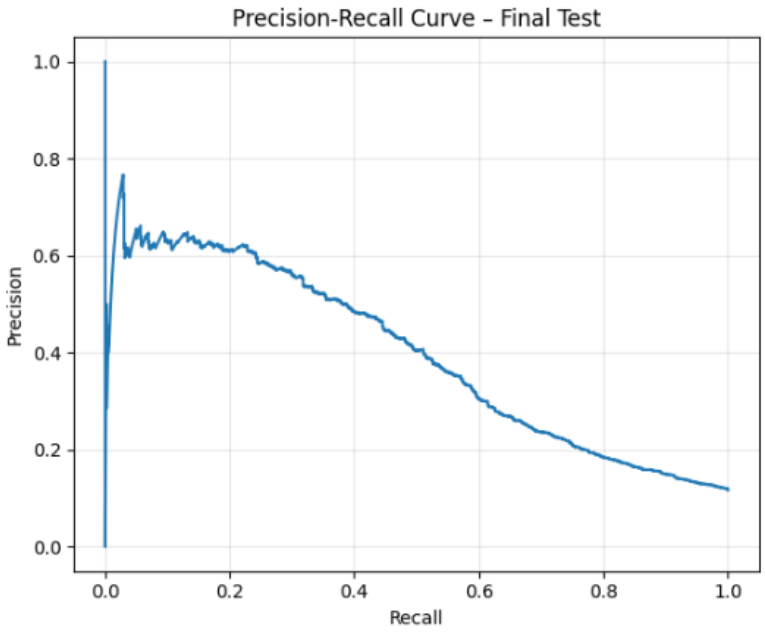


- Grafik Loss dan Accuracy Curve pada Model

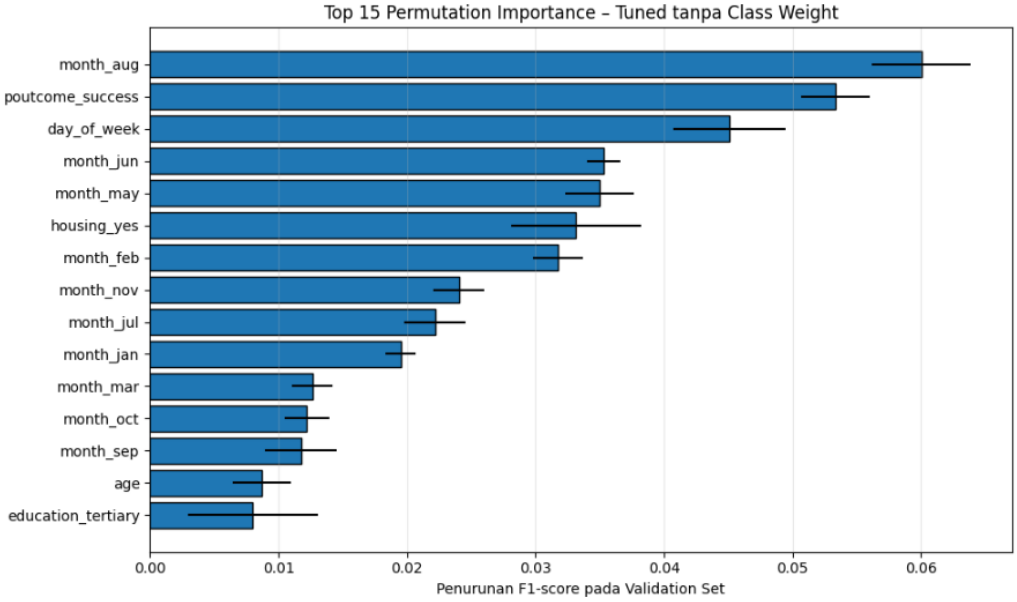


- Grafik Performa Model





- Permutation Importance



- Ablation Study

	Skenario	Threshold	Accuracy	Precision_yes	Recall_yes	F1_yes	ROC_AUC	PR_AUC
0	Tanpa empat fitur engineered @ 0.50	0.5	0.8953	0.6352	0.2459	0.3545	0.7745	0.4310
1	Dengan empat fitur engineered @ 0.50	0.5	0.8937	0.6241	0.2282	0.3343	0.7729	0.4391

10. Daftar Pustaka

[1] S. Moro, P. Cortez, and P. Rita, "A data-driven approach to predict the success of bank telemarketing," *Decis. Support Syst.*, pp. 22–31, 2014, doi: 10.1016/j.dss.2014.03.001.

[2] S. Moro, P. Rita, and P. Cortez, "Bank Marketing," UCI Machine Learning Repository. Accessed: Nov. 02, 2026. [Online]. Available: https://archive.ics.uci.edu/dataset/222/bank+marketing

[3] Scikit-learn developers, "StandardScaler," Scikit-learn API Reference. Accessed: Nov. 02, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html

[4] Scikit-learn developers, "compute_class_weight," Scikit-learn API Reference. Accessed: Nov. 02, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.utils.class_weight.compute_class_weight.html

[5] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," PMLR, 2015. Accessed: Nov. 02, 2026. [Online]. Available: https://proceedings.mlr.press/v37/ioffe15.html

[6] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014, Accessed: Nov. 02, 2026. [Online]. Available: https://jmlr.org/papers/v15/srivastava14a.html

[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2015. Accessed: Nov. 02, 2026. [Online]. Available: https://arxiv.org/abs/1412.6980

[8] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS One*, vol. 10, no. 3, 2015, doi: 10.1371/journal.pone.0118432.

[9] Scikit-learn developers, "permutation_importance," Scikit-learn API Reference. Accessed: Nov. 02, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.inspection.permutation_importance.html

[10] TensorFlow, “tf.keras.callbacks.EarlyStopping,” TensorFlow API Reference. Accessed: Nov. 02, 2026. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/EarlyStopping

[11] TensorFlow, “tf.keras.callbacks.ReduceLROnPlateau,” TensorFlow API Reference. Accessed: Nov. 02, 2026. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/ReduceLROnPlateau

[12] TensorFlow, “tf.keras.metrics.AUC,” TensorFlow API Reference. Accessed: Nov. 02, 2026. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/metrics/AUC

[13] Streamlit, “st.form,” Streamlit API Reference. Accessed: Nov. 02, 2026. [Online]. Available: https://docs.streamlit.io/develop/api-reference/execution-flow/st.form

B. Tautan

No.	Detail	Tautan
1	Google Colab Notebook	https://colab.research.google.com/drive/1PLJNb7z1_kkQlx09fGuEj3jJSAJ_7XzZ?usp=sharing
2	GitHub	https://github.com/VincentArfan/Prediksi-Deposito-Bank
3	Deploy Aplikasi dengan Streamlit	https://prediksi-deposito-bank-6nuk6vumea3pydkff6gvrq.streamlit.app/

Tabel 13 Daftar Tautan